

CAN VISION-LANGUAGE MODELS LEARN TO EXECUTE CLASSICAL SEARCH ALGORITHMS?

Anonymous authors

Paper under double-blind review

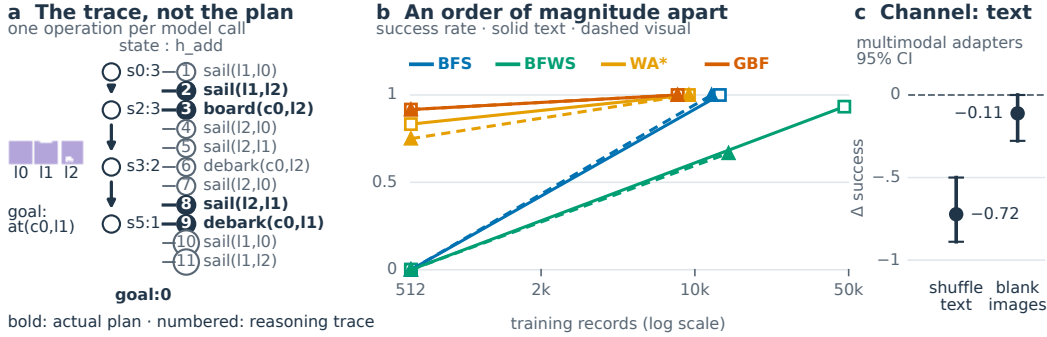


Figure 1: **Search traces as the supervision target, not plans:** (a) One ferry task: the usual plan-level supervision stops at the plan (bold, four actions); we supervise the search behind it (numbered, 11 operations) (b) colour marks the algorithm (BFS, BFWS, weighted A*, greedy best-first) and line style the observation (solid text, dashed visual); all four are learnable, but BFS and BFWS need far larger corpora. (c) corrupting the text of the observation costs far more success than the images.

ABSTRACT

The automated planning community has probed the planning ability of large language models by fine-tuning them on planning-problem corpora, but the training target is almost always the *plan* (i.e., the action sequence a planner returns), rather than the search that found it. Yet classical planning offers many search algorithms, and how well a model learns to execute them from its reasoning trace is still an open question. As embodied agents increasingly plan from what they see, the question extends to vision-language models (VLMs). We fine-tune a VLM on the search traces of four classical algorithms, breadth-first search (BFS), best-first width search (BFWS), and greedy and weighted- A^* best-first search with the additive heuristic, across 12 PDDL domains with text, image, and combined observations. We find that all four algorithms are learnable, but at very different data costs: BFS and BFWS require over an order of magnitude more training data than the additive variants to reach the same performance. Across observation types, text encodes the learned knowledge: shuffling the text lowers success by up to 0.94, while blanking the images costs at most 0.11. Where plan-trained models have been reported to degrade under symbol renaming, trace-trained models stay robust to renaming, restyling, and scale-up. Training on width-based traces even transfers beyond planning, raising logical-reasoning accuracy on FOLIO from 0.600 to 0.660. Together, these results establish search traces as a practical supervision target for planning research, alongside plans.

1 INTRODUCTION

The automated planning community has probed the planning ability of large language models by fine-tuning them on planning problems, and the training target is almost always the plan, the action sequence a planner returns, rather than the search that found it (Pallagani et al., 2022; Rossetti et al., 2024; Bohnet et al., 2024; Belcamino et al., 2026). A plan records the outcome of a search but not the procedure behind it, and there are signs that plan-trained models rely partly on the surface form of their training plans. Belcamino et al. (2026) report that a model trained on 40,000 plans

loses significant accuracy when symbols are anonymized or plans are serialized more compactly, although neither change alters what the plans mean.

Classical planners owe their domain independence to search: one algorithm solves tasks in any PDDL domain, and search algorithms differ mainly in how they order the states still to be expanded. A search trace records every one of these steps, so training on traces targets the procedure itself. Prior work has trained models on search traces (Lehnert et al., 2024; Saha et al., 2025), but these studies centre on A* in mazes, Sokoban, and Blocksworld, so how the choice of search algorithm affects what a model learns remains underexplored. The question extends to vision-language models (VLMs), because recent embodied agents use VLMs to perceive a scene and plan from it end to end (Merler et al., 2026; Xu et al., 2026; Hao et al., 2026), and how the observation modality affects learning to search is unknown.

We study these questions by fine-tuning a VLM on the search traces of four algorithms that cover three ways of ordering a search. Breadth-first search (BFS) is blind: it expands states in the order they were generated. Best-first width search (BFWS) (Lipovetzky & Geffner, 2017) orders states by novelty, preferring states that make a fact or pair of facts true for the first time, and breaks ties by the number of unachieved goals. Greedy best-first search and weighted A* order states by the additive heuristic h_{add} (Bonet & Geffner, 2001), an estimate of the distance to the goal, and differ in whether the cost of the path so far also counts. The model executes the search itself, one operation at a time, and learns from the full trace rather than the plan alone (Figure 1a). We train and test it on 12 PDDL domains, with each state shown as text, as an image, or as both, which we call text, visual, and multimodal observations.

Our results show that a VLM can learn to execute every one of these algorithms, and two findings run against a natural expectation: the simplest algorithm, blind BFS, is among the most expensive to learn, and the images a VLM receives contribute little to what it learns.

The data cost of learning depends strongly on the algorithm (Section 5.1, Figure 1b). Weighted A* and greedy best-first search reach success rates of 0.833 and 0.903 from a small training set, whereas BFS and BFWS remain at 0 and reach 1.0 and 0.933 only with over an order of magnitude more data.

The learned search relies on the text rather than the images (Section 5.2, Figure 1c). When the model sees both, shuffling the text lowers its success by 0.722, whereas blanking or degrading the image lowers it by at most 0.111.

The learned heuristic search is also insensitive to the surface representation of a task (Section 5.3). Plan-trained models have been reported to show strong sensitivity to surface representations, losing significant accuracy under symbol anonymization that preserves plan semantics (Belcamino et al., 2026), but our trace-trained models show no such drop. Their success stays at 0.967 under object renaming and 0.900 under name compression and image restyling, holds at 0.867 on scaled-up tasks, and falls to 0.633 only when the initial state is shifted.

Whether search skills learned in one setting carry over to reasoning more generally is a question of wide interest (Gandhi et al., 2024; Opedal et al., 2026), and here width-based traces behave differently from the rest. Training on BFWS traces raises accuracy on the FOLIO logical-reasoning benchmark (Han et al., 2022) from 0.600 to as much as 0.660 (Section 5.4), whereas training on the other three algorithms leaves it at or below 0.605, and no trained model changes GSM8K or HumanEval accuracy by more than 0.04 (Cobbe et al., 2021; Chen et al., 2021). This suggests that width-based search teaches a skill that reaches beyond planning, which may be worth pursuing as a research direction in its own right.

We make three contributions.

- **A study of learning search from traces.** We fine-tune a VLM on the traces of blind, width-based, and heuristic search over 12 PDDL domains, with text, visual, and multimodal observations.
- **What learning depends on.** The data cost of learning a search algorithm varies by over an order of magnitude across algorithms, and the learned search relies on the text rather than the images.
- **What the learned search carries.** Trace-trained heuristic search is insensitive to the surface representation of a task, and width-based traces point to a reasoning skill that reaches beyond planning.

2 RELATED WORK

Learning to plan from plans. Language models trained for classical planning output plans, either from scratch (Rossetti et al., 2024) or by fine-tuning pretrained ones on planner-generated plans (Pallagani et al., 2022; Bohnet et al., 2024; Li et al., 2024; Belcamino et al., 2026), sometimes reasoning along a candidate plan (Verma et al., 2025; Emami et al., 2026). Belcamino et al. (2026) report sensitivity to surface representations: symbol anonymization causes significant drops despite preserving plan semantics. We supervise the search instead, including states that never reach the plan.

Learning from search traces. Models have also learned from BFS, A*, and other search traces in mazes, Sokoban, Countdown, code repair, and deductive proofs (Yang et al., 2022; Lehnert et al., 2024; Su et al., 2025; Gandhi et al., 2024; Moon et al., 2025; Opedal et al., 2026). Search-former beats plan-only training with a tenth of the data (Lehnert et al., 2024), and the closest work, System-1.x, fine-tunes a pretrained LLM on A* traces in mazes and Blocksworld and shows that it also works with BFS or DFS traces in mazes (Saha et al., 2025). Three questions remain open across these studies. They cover a few puzzle or reasoning tasks and do not compare what different algorithms cost to learn; none learns width-based search, a mainstay of classical planning; and each evaluates the learned search only within the task it was trained on. We compare four algorithms, including BFWS, across 12 PDDL domains, and test transfer to logic, math, and code.

Planning from images. VLMs have been evaluated as planners and as grounders for a symbolic planner (Merler et al., 2026), fine-tuned to simulate action outcomes from images for a PDDL planner (Hao et al., 2026), and trained to plan through image sequences, outperforming text reasoning on visual navigation tasks (Xu et al., 2026). None of these models is trained to execute a search, and none tests which input channel a trained model relies on when the same task arrives as text, images, or both. We train a VLM to execute the search and corrupt each channel in turn: the learned search relies on the text (Section 5.2), echoing findings on language priors (Zhu et al., 2020).

3 LEARNING TO EXECUTE SEARCH FROM TRACES

We fine-tune a vision-language model to act as a **Search Process Policy**, which executes a declared search algorithm one step at a time instead of writing a plan. An episode fixes a task x in a deterministic grounded PDDL domain, a declared algorithm A , an observation type m , and a decision budget B . At each step t , the policy reads an observation o_t and emits one **Typed Search Operation**, a structured record naming an operation of A and every operand that determines exploration. A **Trusted Search Runtime** checks the operation against the **Algorithm Invariant** of A , the ordering and bookkeeping rules that define it (Appendix B), and applies it. The runtime never fills in a missing operand or repairs an emission, and the first invalid step ends the episode (Appendix C). The frontier, visited set, and BFWS novelty tables live in the runtime’s **Search Memory**, so o_t is the policy’s only view of the search.

In every algorithm, the runtime expands the head of its frontier and lists that state’s grounded successors as candidates (Figure 2). A valid step names this state and inserts one listed successor, and a retire step closes the expansion once none is left. Under BFS, the policy picks a candidate flagged as unvisited and emits the bookkeeping of a first-in, first-out (FIFO) queue: its position at the back of the queue and whether this step removes the head. Under BFWS (Lipovetzky & Geffner, 2017), each candidate is flagged as a duplicate or not, and each new one carries the runtime’s evaluation of its novelty, its number of unachieved goals, and the frontier position this priority implies. The policy picks a non-duplicate candidate and copies that position.

The two additive variants are greedy best-first search (Bonet & Geffner, 2001), ordered by h_{add} , and weighted A* (Hart et al., 1968; Pohl, 1970), ordered by $g + 3h_{\text{add}}$. Each candidate row lists g , h_{add} , and the priority, all computed by the runtime, which also keeps the priority heap and its tie-break rule. The policy names the expanded state and one candidate not yet submitted, and emits no heuristic value or frontier position. In all four algorithms, each expansion thus submits every new successor, and the observation supplies the values that fix the frontier order. We call this setting the **enumeration contract**.

The three observation types differ only in how the task’s facts reach the model. The text observation lists the objects, static facts, initial and current states, and goal. The visual observation replaces

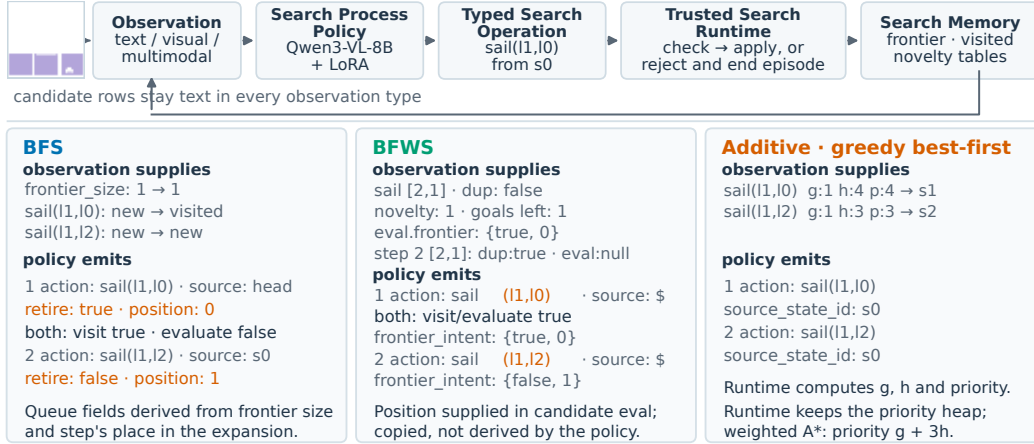


Figure 2: **What one valid step contains under each algorithm.** Top: the policy reads an observation and emits one typed operation; the runtime checks and applies it, and the first invalid operation ends the episode. Bottom: the first two steps on the ferry task of Figure 1a. Grey fields are copied from the observation; orange fields must be derived by the policy.

these facts with images: scenes of the initial and current states drawn by the external Planimation renderer¹, and pages that render the static facts and goal. The multimodal observation gives both. In all three, the candidates, frontier summary, and recent operations stay in text, so only state facts become images.

We train on the search trace, not the plan. Each training record, a **Search-Trace Segment**, pairs the observation at one step of a replay-verified trace of the exact reference (the declared algorithm run by the runtime) with the operation emitted there. The loss is token-level cross-entropy on that operation. A plan target would supervise only the returned action sequence, not the search steps behind it.

Under the enumeration contract, the runtime decides which state is expanded next. An appendix study lets the policy make this choice from rendered scenes alone, under a choice-frontier contract for the two additive variants (Appendix Q.1).

4 EXPERIMENTAL DESIGN

We fine-tune Qwen3-VL-8B-Instruct (Bai et al., 2025) (revision 0c351dd0) with low-rank adaptation (LoRA) (Hu et al., 2022) and repeat two algorithms on a second backbone, InternVL3.5-8B (Wang et al., 2025). The main grid, which we call the expanded baseline, covers 24 tasks from 12 PDDL domains, and each of its 12 cells pairs one algorithm with one observation type. The large-corpus runs described below were evaluated on separate development panels, which differ from the main grid in tasks and size (Appendix E).

The main recipe is identical for all four algorithms: 512 training records, one epoch, 16 optimizer updates, and training seed 17. To probe how much data BFS and BFWS need, we also report earlier large-corpus development runs. BFS trained on 12,994 records for three epochs and was evaluated on 15 development tasks. BFWS trained on 47,780 records for two epochs and was evaluated on a different 15-task panel. A visual run trained all four algorithms on 43,876 records for two epochs and was evaluated on 42 task groups. These runs differ from the main grid in data, schedule, and panel, so they indicate the data cost and are not a matched-size comparison.

Three controls frame each learned cell. The pretrained base runs without an adapter. The exact reference runs the declared algorithm in the runtime and reaches the goal on every main-grid task. Random-valid is oracle-assisted: at each step it reads the runtime’s list of valid operations and picks one uniformly at random. It therefore bounds what validity alone achieves and is not a competing method. In the additive cells, the runtime supplies every candidate and orders the frontier itself, so

¹<https://github.com/planimation/backend>

every valid submission order yields the same search and random-valid necessarily matches the exact reference (Appendix P).

An episode succeeds when it reaches the goal with every step valid and within its budget. Budgets count decisions and charge invalid operations. Adapters may use twice the exact reference’s decision count, and the pretrained base is priced at one call. An episode counts only after independent replay. Contrasts use a paired whole-instance bootstrap (Appendix F).

Four analyses follow the main grid. An observation-corruption suite shuffles or masks the text, or blanks or degrades the images, in the additive adapters’ observations at test time (Appendix K). Robustness variants apply five families of task changes to the additive adapters: object renaming, name compression, scene restyling, scale-up, and shifted initial states. Transfer evaluates the main-grid adapters on FOLIO (Han et al., 2022) (200 items), GSM8K (Cobbe et al., 2021) (200), and HumanEval (Chen et al., 2021) (164), using McNemar tests (McNemar, 1947) against the base with Holm correction (Holm, 1979). The second backbone reruns BFS and greedy best-first search on a 12-task panel.

All results are development-stage. Learned main-grid cells use one training seed, except two cells replicated over three training seeds, and the frozen 45-task final evaluation has not been run. Every executed run reconciles with its ledger and independent replay (Appendix T, Table 26).

5 RESULTS

This section reports four findings. Section 5.1 shows that all four algorithms can be learned from traces, at very different data costs. Section 5.2 shows that the learned execution relies on the text of each observation, Section 5.3 that it keeps working under task variations, and Section 5.4 that BFWS traces bring a small, not statistically significant gain in logical reasoning. Two controls appear throughout. Random-valid emits a uniformly drawn valid operation at each decision, and the exact reference runs the declared algorithm itself. Both read the runtime’s list of valid operations, so we treat them as oracle-assisted bounds on what valid execution reaches, not as learned abilities (Section 4).

5.1 ALL FOUR ALGORITHMS ARE LEARNABLE, AT DIFFERENT DATA COSTS

With 512 training records per adapter, fine-tuning teaches the two additive variants but not BFS or BFWS (Table 1, top). Each record is one search decision taken from a replay-verified reference trace. The greedy best-first and weighted-A* adapters solve 125/144 task-observation entries, whereas the pretrained base solves none and mostly fails its first decision on output format (Table 2). The BFS and BFWS adapters solve 0/24 tasks under every observation.

Random-valid solves every additive task, which is the expected behavior of this control. In the additive cells the runtime supplies the grounded candidates and orders the priority queue, so on these tasks any valid emission reproduces the reference’s search (Appendix P). The additive result therefore shows that the adapters learned to execute the algorithm’s steps validly, and random-valid marks the ceiling that valid execution reaches.

BFS and BFWS become learnable when the corpus grows by more than an order of magnitude, although we tested no corpus size between 512 and about 11,900 records, so the minimum that suffices is unknown (Table 1, bottom). Text BFS adapters trained on 12,994 records for three epochs solve every task of a 15-task development panel at each of five training seeds, and random-valid also solves every task there. A text BFWS adapter trained on 47,780 records for two epochs reaches 0.933 on another 15-task panel, against 0.467 for random-valid, and the paired bootstrap 95% lower bound on its gain is 0.2. A visual run on 43,876 records across the four algorithms, including 11,911 for BFS and 14,225 for BFWS, solves 75/75 BFS episodes (random-valid 53/75) and 50/75 BFWS episodes (random-valid 39/75), and both additive adapters solve every episode, as random-valid does. These runs use their own development panels and schedules, so they are not a matched-size comparison with the main grid. They establish that BFS and BFWS can be learned from traces, and that 512 records are not enough under our recipe.

One explanation, which we have not tested, is that BFS and BFWS steps each ask more of the policy than an additive step does, in different ways. The runtime flags each candidate as unvisited

Table 1: **All four algorithms are learnable, but BFS and BFWS need far larger corpora.** Top: the main grid, with one adapter per algorithm and observation trained on 512 records for one epoch. Entries count solved tasks out of 24 from 12 domains. Base and controls do not vary with observation, and the total row counts all three observations. Bottom: large-corpus development runs, each on its own development panel and schedule, reported as success rates. These panels differ from the main grid and from each other, so the two blocks are not a matched-size comparison. Random-valid and exact are oracle-assisted bounds that read the runtime’s valid-operation list.

Algorithm	Base	SFT			Random-valid	Exact
		text	visual	multimodal		
BFS	0	0	0	0	15	24
BFWS	0	0	0	0	17	24
Weighted A* ($w=3$)	0	20	18	22	24	24
Greedy best-first	0	22	22	21	24	24
Additive total (of 144)		0	125		144	144

Algorithm	Obs.	Records	Epochs	Development panel	SFT	Random-valid	Exact
BFS	text	12,994	3	15 tasks	1.000	1.000	1.000
BFWS	text	47,780	2	15 tasks	0.933	0.467	1.000
BFS	visual	11,911	2	42 task groups	1.000	0.707	1.000
BFWS	visual	14,225	2		0.667	0.520	1.000
Weighted A* ($w=3$)	visual	9,398	2		1.000	1.000	1.000
Greedy best-first	visual	8,342	2		1.000	1.000	1.000

or duplicate and computes the BFWS novelty, goal count, and implied frontier position. A BFS step must still pick an unvisited candidate and emit first-in, first-out bookkeeping, meaning the candidate’s queue position and whether the step removes the head. A BFWS step must follow a larger operation and observation format, with object arguments written as indices and more fields to copy, including the frontier position that the runtime’s priority implies. An additive step only names the expanded state and one listed candidate, with no position or heuristic value (Section 3). The 512-record failures split the same way. Of the 72 BFS failures, 68 are well-formed steps that name a listed successor but misstate the queue bookkeeping, and the other 4 retire the frontier too early. Every BFWS failure is a first operation that does not follow the BFWS operation format (Figure 4, Appendix J).

The main-grid pattern holds across training seeds and on a second backbone. Retraining the greedy multimodal cell with seeds 17, 29, and 71 solves 21, 21, and 22 of 24 tasks (Table 3). InternVL3.5-8B (Wang et al., 2025), trained with the same 512-record recipe and evaluated on 12-task subsets of the main panel, solves 31/36 greedy best-first entries and 0/36 BFS entries, and its base solves none in either cell (Appendix I). Because this backbone shares the Qwen3 language-model family with our primary one, the replication varies the vision encoder and multimodal recipe rather than the language model.

5.2 TEXT CARRIES THE LEARNED EXECUTION

The trained controllers depend on the text of their observations and hardly on the images. We corrupted the observations of the frozen additive adapters at test time, without retraining, on 9 tasks under both additive algorithms (Figure 1c, Table 9). Shuffling the text while leaving its operations copyable changes success by $-0.944 [-1.000, -0.833]$ in the text arm and by -0.722 in the multimodal arm. Blanking or degrading the images changes success by at most -0.111 , and every visual interval includes zero. Masking the text also collapses success, but it does so by breaking the output format, since every masked episode emits only invalid operations. The channel claim therefore rests on shuffling.

This finding has a specific scope. In all three observation types of the main grid, the runtime supplies the grounded candidate list and the search memory as text, and only the state and goal become images. Image-only arms that drop every textual aid except the task interface and this candidate menu still solve 18/18 tasks, and blanking every image of the arm that also sees rendered history leaves its success unchanged (Appendix M). For planning, the result means that the trained controllers read

the textual serialization of the search, so rendering the state does not yet add information that the model uses. It does not show that images are useless for planning in principle.

5.3 ROBUSTNESS TO TASK VARIATIONS

The learned additive search keeps working when tasks are varied. We evaluated the six main-grid additive adapters, two algorithms under three observations, on 25 derived tasks drawn five each from five variation families and frozen before execution. They solve 128/150 episodes, while the pretrained base solves 0/150 (Appendix I). By family, they solve 29/30 with renamed objects, 27/30 with compressed names, 27/30 with restyled renderings, and 26/30 on scaled-up instances. Tasks that start from a shifted initial state are the hardest, at 19/30. Random-valid (750/750) and the exact reference (150/150) saturate, so these counts show that learned execution survives the variations, not that it exceeds the controls. Each family holds five tasks, so per-family counts are descriptive. We tested only the two additive adapters on variants, not BFS or BFWS.

These variations resemble those under which a plan-trained model has been reported to degrade. Belcamino et al. (2026) fine-tune a 1.7B-parameter language model on domain-problem-plan tuples and report significant drops under instance-wise symbol anonymization and compact plan serialization. The two setups differ. Their model writes a whole plan, while ours emits one search step at a time from grounded candidates that the runtime supplies. We ran no plan-only baseline, and our controls saturate, so the two results sit side by side and cannot be ranked against each other.

5.4 TRANSFER BEYOND PLANNING

Training on BFWS traces raises logical-reasoning accuracy under all three observations, a consistent but not statistically significant gain. We evaluated all 12 main-grid adapters zero-shot on frozen subsets of FOLIO (Han et al., 2022) (200 items), GSM8K (Cobbe et al., 2021) (200), and HumanEval (Chen et al., 2021) (164). On FOLIO the BFWS adapters score 0.660, 0.645, and 0.645 (text, visual, multimodal) against 0.600 for the base, while the other adapters score between 0.575 and 0.605. GSM8K accuracy stays between 0.945 and 0.965 for every adapter (base 0.955), and HumanEval between 0.756 and 0.817 (base 0.780). No adapter-versus-base comparison survives Holm correction over all 36 comparisons (Holm, 1979). The smallest raw McNemar p (McNemar, 1947) is 0.023, for the text BFWS adapter on FOLIO, and the minimum adjusted p is 0.82 (Table 8, Appendix I). A leakage screen finds no content overlap between the benchmarks and the planning corpus.

These are the 512-record adapters of the main grid, so the BFWS adapter that gains on FOLIO solves 0/24 planning tasks. The FOLIO gain thus appears without any planning success at that budget. One speculative reading, which we have not tested, is that exposure to the novelty and duplicate checks in BFWS traces helps with logical entailment.

6 DISCUSSION AND LIMITATIONS

For planning research, our results suggest that search traces are a practical supervision target alongside plans. A model trained on traces learns all four algorithms, but at data costs that differ by more than an order of magnitude: 512 records for the additive variants, against 12,994 and 47,780 records in development runs for BFS and BFWS (Section 5.1). The 512-record failures suggest an untested explanation. An additive step only names a listed candidate, and the runtime computes every value that orders the frontier. BFS adapters emit well-formed steps that name a listed successor but get the queue bookkeeping wrong: the back-of-queue position or the head-removal flag. Every BFWS adapter failure is instead a malformed first operation, so the model apparently must first learn the larger BFWS operation and observation format, with more fields and indexed arguments, before its bookkeeping matters. Learning these formats and this bookkeeping from less data is the first open problem.

The trained controllers read the textual serialization. For the additive adapters, shuffling the text collapses success, while blanking or degrading the images barely changes it (Section 5.2). This does not show that images are useless in principle, because in every observation type the candidate list stays in text and only state facts become images. Making the image channel carry the decision is the

second open problem. An appendix study lets the model pick which frontier state to expand from rendered scenes alone (Appendix Q). On that study’s solve-versus-budget score, the trained model reaches 0.306, above 0.021 for random choice but far below 0.875 for exact search.

The robustness and transfer results are narrower. The heuristic best-first adapters keep solving tasks under renaming, name compression, restyling, and scale-up, and shifted initial states hurt them most (Section 5.3). Plan-trained models have been reported to degrade when symbols are anonymized (Belcamino et al., 2026), but the setups differ: that model writes a whole plan, whereas ours emits one search step over runtime-supplied candidates. Training on BFWS traces raises FOLIO accuracy under all three observations (Section 5.4), a consistent gain that is not statistically significant, obtained from adapters that solve no planning task at that budget.

Limitations.

- *Stage.* All results are development-stage, and the frozen 45-task final evaluation has not been run.
- *Seeds.* Each main-grid cell uses one training seed, except two cells replicated over three seeds.
- *Data cost.* The data-cost comparison is not matched: the large-corpus runs use different, easier panels and more epochs than the 512-record grid, and random-valid also solves the BFS panel. No BFS or BFWS corpus between 512 and 11,911 records was tested, so the smallest sufficient size is unknown.
- *Runtime.* The runtime supplies grounded candidates as text, so random-valid solves every additive task and success there measures valid execution, not frontier choice.
- *Robustness.* Only the heuristic variants were tested on task variants, and we trained no plan-only baseline.
- *Transfer.* No transfer gain survives Holm correction over 36 comparisons.
- *Backbone.* We study one language-model family at 8B scale, and the second-backbone check (InternVL3.5-8B) uses a reduced 12-task panel.
- *Appendix study.* The choice-frontier study covers only the two additive variants with scene-only observations.

7 CONCLUSION

We asked whether a vision-language model can learn to execute classical search algorithms from their traces. On development panels it learns all four algorithms, at data costs that differ by over an order of magnitude. The learned execution rests on the textual serialization, holds up under task variations for the heuristic variants, and hints at transfer to logical reasoning. Search traces are thus a practical supervision target for planning research, alongside plans. Two problems remain open: learning BFS and BFWS from less data, and making the image channel carry the search decision.

REPRODUCIBILITY STATEMENT

Appendices A, B, and C specify the operation schemas, algorithm invariants, and runtime that check every search step. Appendix E gives the trace corpora, splits, and training recipes, and Appendix F the statistical procedures. The corruption, robustness, transfer, and second-backbone analyses are detailed in Appendices K and I.

Appendix Q.1 specifies the frontier-choice study, whose held-out panels, DAgger ablation, and zero-shot policy are reported in Appendices Q.4.1, Q.4.3, and Q.4.4. Table 26 holds the run ledgers, and Table 27 indexes the pinned evidence paths. Code and evidence bundles will be provided as anonymous supplementary material.

REFERENCES

- Shuai Bai, Yuxuan Cai, Ruizhe Chen, et al. Qwen3-VL technical report, 2025.
- Valerio Belcamino, Nicholas Attolino, Alessio Capitanelli, and Fulvio Mastrogiovanni. On the generalization gap in LLM planning: Tests and verifier-reward RL, 2026.

- Bernd Bohnet, Azade Nova, Aaron T. Parisi, Kevin Swersky, Katayoon Goshvadi, Hanjun Dai, Dale Schuurmans, Noah Fiedel, and Hanie Sedghi. Exploring and benchmarking the planning capabilities of large language models, 2024.
- Blai Bonet and Héctor Geffner. Planning as heuristic search. *Artificial Intelligence*, 129(1–2):5–33, 2001.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, July 2021.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, October 2021.
- Patrick Emami, Nan Qiang, and Peter Graf. A close look at world model recovery in supervised fine-tuned LLM planners, 2026.
- Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D. Goodman. Stream of search (SoS): Learning to search in language. In *Conference on Language Modeling (COLM 2024)*, 2024. Spotlight. arXiv:2404.03683.
- Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alex Wardle-Solano, Hannah Szabo, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander R. Fabbri, Wojciech Kryscinski, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying, Arman Cohan, and Dragomir Radev. FOLIO: Natural language reasoning with first-order logic, September 2022.
- Yilun Hao, Yongchao Chen, Chuchu Fan, and Yang Zhang. Simulation to rules: A dual-VLM framework for formal visual planning. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*, 2026. URL <https://openreview.net/forum?id=7t1LpQpGlx>. arXiv:2510.03182.
- Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.
- Nathan Herr, Tim Rocktäschel, and Roberta Raileanu. LLM-first search: Self-guided exploration of the solution space. arXiv:2506.05213, 2025.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations (ICLR)*, 2022.
- Lucas Lehnert, Sainbayar Sukhbaatar, DiJia Su, Qinqing Zheng, Paul Mcvay, Michael Rabbat, and Yuandong Tian. Beyond A*: Better planning with transformers via search dynamics bootstrapping, 2024.

- Liam Li and Ameet Talwalkar. Random search and reproducibility for neural architecture search. In *Proceedings of The 35th Uncertainty in Artificial Intelligence Conference*, volume 115 of *Proceedings of Machine Learning Research*, pp. 367–377. PMLR, 2020. URL <https://proceedings.mlr.press/v115/li20c.html>.
- Wenjun Li, Changyu Chen, and Pradeep Varakantham. Unlocking large language model’s planning capabilities with maximum diversity fine-tuning, 2024.
- Nir Lipovetzky and Héctor Geffner. Best-first width search: Exploration and exploitation in classical planning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, pp. 3590–3596, 2017. doi: 10.1609/aaai.v31i1.11027.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- Matteo Merler, Nicola Dainese, Minttu Alakuijala, Giovanni Bonetta, Pietro Ferrazzi, Yu Tian, Bernardo Magnini, and Pekka Marttinen. ViPlan: A benchmark for visual planning with symbolic predicates and vision-language models. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. arXiv:2505.13180.
- Seungyong Moon, Bumsoo Park, and Hyun Oh Song. Learning to better search with language models via guided reinforced self-training. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. arXiv:2410.02992.
- Andreas Opedal, Francesco Ignazio Re, Abulhair Saparov, Mrinmaya Sachan, Bernhard Schölkopf, and Ryan Cotterell. Learning to reason efficiently with a* post-training, 2026. URL <https://arxiv.org/abs/2605.24597>.
- Vishal Pallagani, Bharath Muppasani, Keerthiram Murugesan, Francesca Rossi, Lior Horesh, Biplav Srivastava, Francesco Fabiano, and Andrea Loreggia. Plansformer: Generating symbolic plans using transformers, 2022.
- Ira Pohl. Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1(3–4):193–204, 1970.
- Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pp. 627–635, 2011.
- Nicholas Rossetti, Massimiliano Tummolo, Alfonso Emilio Gerevini, Luca Putelli, Ivan Serina, Mattia Chiari, and Matteo Olivato. Learning general policies for planning through GPT models. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, volume 34, pp. 500–508. AAAI Press, 2024. doi: 10.1609/icaps.v34i1.31510. URL <https://ojs.aaai.org/index.php/ICAPS/article/view/31510>.
- Swarnadeep Saha, Archiki Prasad, Justin Chih-Yao Chen, Peter Hase, Elias Stengel-Eskin, and Mohit Bansal. System-1.x: Learning to balance fast and slow planning with language models. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025. arXiv:2407.14414.
- Dhruv Shah, Michael Robert Equi, Błażej Osiński, Fei Xia, Brian Ichter, and Sergey Levine. Navigation with large language models: Semantic guesswork as a heuristic for planning. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pp. 2683–2699, 2023. URL <https://proceedings.mlr.press/v229/shah23c.html>.
- Katharina Stein, Daniel Fišer, Jörg Hoffmann, and Alexander Koller. Automating the generation of prompts for LLM-based action choice in PDDL planning. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, volume 35, pp. 250–259, 2025.

- DiJia Su, Sainbayar Sukhbaatar, Michael Rabbat, Yuandong Tian, and Qingqing Zheng. Dual-former: Controllable fast and slow thinking by learning with randomized reasoning traces. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025. URL <https://openreview.net/forum?id=bmbRCRiNDu>. arXiv:2410.09918.
- Pulkit Verma, Ngoc La, Anthony Favier, Swaroop Mishra, and Julie A. Shah. Teaching LLMs to plan: Logical chain-of-thought instruction tuning for symbolic planning, 2025.
- Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. ScienceWorld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP 2022)*, pp. 11279–11298, 2022. URL <https://aclanthology.org/2022.emnlp-main.775/>.
- Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, et al. InternVL3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency, August 2025.
- Yi Xu, Chengzu Li, Han Zhou, Xingchen Wan, Caiqi Zhang, Anna Korhonen, and Ivan Vulić. Visual planning: Let’s think only with images. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*, 2026. arXiv:2505.11409.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, May 2025.
- Mengjiao Yang, Dale Schuurmans, Pieter Abbeel, and Ofir Nachum. Chain of thought imitation with procedure cloning. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pp. 36366–36381, 2022. doi: 10.52202/068431-2635. arXiv:2205.10816.
- Xi Zhu, Zhendong Mao, Chunxiao Liu, Peng Zhang, Bin Wang, and Yongdong Zhang. Overcoming language priors with self-supervised learning for visual question answering. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, pp. 1083–1089, 2020. doi: 10.24963/ijcai.2020/151. URL <https://www.ijcai.org/proceedings/2020/151>.

Appendix roadmap. The appendices first specify the interface and the data, then report the trace-learning results in detail, and then document the audits and the separate choice-frontier study. Appendices A, B, and C define the operations, invariants, and runtime, and show that the runtime never chooses or repairs on the policy’s behalf. Appendix D states what each observation exposes, Appendix E gives the corpora, splits, and training recipes, and Appendix F states what an interval licenses. Appendices G to N report the trace-study results: the primary matrix and its paired contrasts, the secondary panels with robustness, transfer, and the second backbone, the failure mechanisms, observation corruption, menu manipulation, the image-only arms, and seed replication. Appendix O shows illustrative search episodes. Appendix P explains why random-valid saturates the additive cells, Appendix Q reports the choice-frontier study, and Appendix R audits two published interfaces. Appendices S and T trace each run to its receipt and ledger, and Appendix V compares the two decision contracts.

A OPERATION SCHEMAS

This appendix gives the operation vocabulary, validity checks, and rejection kinds of each declared algorithm, for readers who need the exact interface behind the Typed Search Operation definition.

The schemas enforce one rule: every operand that determines exploration must come from the policy (Section 3). Each declared algorithm defines a vocabulary of operation types τ , and a policy

emission $u_t = (\tau_t, \theta_t)$ pairs a type from that vocabulary with an operand vector θ_t carrying every exploration-determining argument. The runtime rejects any emission whose type falls outside the declared vocabulary or whose operand vector omits a required field, so it never supplies a missing operand itself.

BFS. The primary operation is node expansion. Its operands name the source state (the current frontier head), the grounded successors selected for insertion, the action producing each successor, and the enqueue position for each. The runtime checks that the named source matches the FIFO frontier head, that each successor results from an applicable grounded action in the source state, that the successor is unvisited, and that each insertion targets the FIFO tail. A goal-test operation names the state to test. The BFS pilot corpus exposes the facts these checks use, namely exact grounded successor candidates and full visited membership at each step.

BFWS (complete unpruned). Expansion operands carry the novelty value and goal-count priority of the source state alongside the fields above. Insertion operands carry each candidate’s duplicate membership, novelty, and priority. The runtime checks that the selected source has the declared best priority, that duplicate membership is correctly reported, and that insertions follow the declared ordering. The BFWS corpus exposes the facts these checks use, namely candidate duplicate, novelty, priority, insertion, and enqueue facts, at every training observation.

Additive best-first. Operands carry the source state’s accumulated path cost g , heuristic estimate h_{add} , and composite value $f = g + w \cdot h_{\text{add}}$ with $w = 3$ for the weighted variant, or $f = h_{\text{add}}$ for the greedy variant. Insertion operands carry per-successor g , h_{add} , and f . The runtime checks that the expanded state has the minimum f in the frontier under the declared tie-breaking rule and that all arithmetic is consistent.

Validity preconditions. Every emission passes through the same ordered sequence of checks, and the first check it fails rejects it with that kind and ends the episode. Each recorded rejection kind therefore names the earliest check an emission failed. The checks run in the following order.

1. **Parse.** The raw output must parse into a well-formed Typed Search Operation with a recognized type and complete operand vector. A failure is recorded as `malformed_output`.
2. **Source identity.** The named source state must match the state the algorithm expects. A failure is recorded as `source_state_mismatch`.
3. **Action applicability.** Every grounded action in the operands must be applicable in the source state. A failure is recorded as `inapplicable_grounded_action`.
4. **Successor schema.** Any named successor state must satisfy the domain schema. A failure is recorded as `schema`.
5. **Successor effects.** The named successor must match the declared effects of the applied action. A failure is recorded as `effect`.
6. **Static context.** Referenced predicates, objects, and constants must be consistent with the task instance. A failure is recorded as `static_context`.
7. **Frontier discipline.** The frontier manipulation must respect the declared ordering (FIFO for BFS, priority for BFWS, f -order for additive best-first). A failure is recorded as `invalid_frontier_operation`.
8. **Algorithm invariant.** Any remaining algorithm-specific constraint not covered above. A failure is recorded as `other_invariant_violation`.

Rejections fall into two categories, format failures and search failures. *Malformed-output* rejections never reach the operation checker, whereas *post-parse rejections* parse but fail a runtime check. The oracle-assisted random-valid control emits only runtime-accepted operations, so its single failure kind is `expansion_budget_exhausted`, recorded when the frozen episode budget runs out before a goal is reached.

Rejection counts. The base model fails mostly on format, while learned policies, pooled over learned arms, fail mostly after parsing, on algorithm invariants (Table 2). The pooled profile does not hold for every algorithm. All 72 learned `malformed_output` failures come from the 512-record

Table 2: **Base failures are mostly malformed output, and learned failures, pooled over learned arms, are mostly post-parse invariant violations.** The 512-record BFWS adapters are the exception and fail on format (Appendix J). Arm labels give failures/episodes where both counts exist; shares use failures within each class.

Arm class	Rejection kind	Failures	Share
Base (447/447)	malformed_output	379	84.8%
	invalid_frontier_operation	67	15.0%
	other_invariant_violation	1	0.2%
Learned (407/906)	other_invariant_violation	229	56.3%
	malformed_output	72	17.7%
	inapplicable_grounded_action	56	13.8%
	effect	27	6.6%
	schema	16	3.9%
	invalid_frontier_operation	5	1.2%
	source_state_mismatch	1	0.2%
	static_context	1	0.2%
Oracle (51 failures)	expansion_budget_exhausted	51	100.0%

BFWS adapters, which fail on format (Appendix J). The counts come from the failure-mechanism calibration, which classified all 905 failures across 2,910 replay-verified episodes with zero unclassified. The `effect`, `schema`, and `static_context` kinds arise in successor-prediction episodes, where the model generates a dynamic-state successor.

B ALGORITHM INVARIANT DEFINITIONS

This appendix states the invariant the runtime enforces for each declared algorithm, which Section 3 defines only in outline.

The Trusted Search Runtime checks the following invariants on every operation. Because each invariant constrains a single step, stepwise validity and invariant compliance do not establish termination, completeness, optimality, or episode success (Section 3).

BFS. BFS keeps its frontier as a FIFO queue, so every expansion must select the queue head as its source and every insertion must target the queue tail. A state already in the visited set must not be re-inserted. This invariant is sufficient for completeness in finite state spaces with unit-cost transitions.

BFWS (complete unpruned). The frontier is a priority queue ordered first by novelty rank (lower is better), then by goal count among states with equal novelty. No generated successor is pruned. A successor whose state already appears in the frontier or visited set is not re-inserted.

Additive best-first, weighted ($w = 3$). The frontier is a priority queue ordered by $f(s) = g(s) + 3 \cdot h_{\text{add}}(s)$, where $g(s)$ is accumulated path cost and $h_{\text{add}}(s)$ is the additive heuristic. Among equal- f states the lower generation serial is expanded first. The expanded state is the heap head. A successor with lower g than an existing frontier entry for the same state reopens that entry with improved cost.

Additive best-first, greedy. The frontier is ordered by $h_{\text{add}}(s)$ alone, with no path-cost component, then by ascending generation serial among ties. The expanded state is the heap head. Reopen and duplicate rules follow the weighted variant. The executed controller’s serial assignment is what makes the enumeration contract insensitive to submission order. In both additive settings, `BestFirstController.start_expansion` assigns serials before submissions, in the deterministic `PDDLStateAuthority.applicable_actions` order (sorted by `GroundedAction.serialize`). The `BestFirstController.apply_operation` heap orders entries by (priority, serial, state identifier, path cost), and `finish_expansion` requires the full candidate set. Submission permutations therefore leave frontier order unchanged on the enumeration contract. A runtime assigning serials in submission order would not be submission-

order invariant and would not show this identity, so the identity is a property of this runtime (Appendix P.1).

C TRUSTED SEARCH RUNTIME

This appendix gives the runtime’s guarantees and its step and episode pseudocode, for readers checking that the runtime never chooses or repairs on the policy’s behalf.

The runtime provides three guarantees that hold across all declared algorithms.

1. **No-repair property.** The runtime validates, applies, persists to Search Memory, or rejects each emission. It never chooses omitted operands, reorders candidates, silently repairs an invalid emission, or substitutes a default search decision.
2. **Reject-and-terminate.** On the first rejected emission the episode terminates with the rejection kind recorded. There is no retry path.
3. **Replay determinism.** Replaying the raw emissions $u_{1:T}$ against the same task instance and the same initial Search Memory reproduces the explored trace. Any step whose outcome depends on runtime repair would violate this property.

The two procedures below make the guarantees concrete. STEP applies the checks of Appendix A in their fixed order, and EPISODE wraps it in a budgeted loop that ends at the first rejection or at the goal.

```

procedure STEP( $u_t, M_{t-1}, A, x$ )
  // Phase 1: Parse the raw emission
  if  $u_t$  does not parse as  $(\tau, \theta)$  with  $\tau \in \text{vocab}(A)$  then
    return REJECT(malformed_output)
  // Phase 2: Validate source identity
  if  $\theta.\text{source} \neq \text{expected}(A, M_{t-1})$  then
    return REJECT(source_state_mismatch)
  // Phase 3: Validate operands against the task and state
  for each grounded action  $a$  named in  $\theta$  do
    if  $a$  is not applicable in  $\theta.\text{source}$  under  $x.D$  then
      return REJECT(inapplicable_grounding_action)
  for each successor state  $s'$  named in  $\theta$  do
    if  $s'$  violates the domain schema then
      return REJECT(schema)
    if  $s' \neq \text{effects}(a, \theta.\text{source})$  then
      return REJECT(effect)
  if any static-context reference in  $\theta$  is inconsistent with  $x$  then
    return REJECT(static_context)
  // Phase 4: Check Algorithm Invariant
  if frontier discipline of  $A$  is violated by  $(\tau, \theta, M_{t-1})$  then
    return REJECT(invalid_frontier_operation)
  if any other invariant of  $A$  is violated then
    return REJECT(other_invariant_violation)
  // Phase 5: Apply and persist
   $M_t \leftarrow \text{apply}(\tau, \theta, M_{t-1})$ 
  return ACCEPT( $M_t$ )

procedure EPISODE( $x, A, \phi_m, B$ )
   $M_0 \leftarrow \text{InitSearchMemory}(A, x)$ 
  for  $t = 1, 2, \dots, B$  do
     $o_t \leftarrow \phi_m(c_t(x, M_{t-1}))$  // build observation
     $u_t \leftarrow \pi(o_t)$  // policy emission
     $r \leftarrow \text{STEP}(u_t, M_{t-1}, A, x)$ 
    if  $r$  is REJECT(kind) then
      return  $(t, \text{FAIL}, \text{kind})$ 
     $M_t \leftarrow r.M$ 
    if GoalTest( $M_t, x.G$ ) = TRUE then
      return  $(t, \text{SUCCESS})$ 

```

return (B , FAIL, expansion_budget_exhausted)

Each call to STEP consumes one unit of the frozen episode budget B whether the emission is accepted or rejected, so invalid operations are charged to the budget (Section 4).

D MODALITY ADAPTERS AND OBSERVATION CONTRACT

This appendix defines the observation adapters and what each one exposes to the policy, which bounds any reading of the modality results.

Every adapter must be decision-sufficient: its observation must expose every fact the exact reference uses to resolve the current step under the declared algorithm. Three fixed Modality Observation adapters are declared. Each maps a decision context c_t (the canonical task context plus the decision-relevant view of Search Memory M_t) to an observation $o_t = \phi_m(c_t)$ presented to the policy, and the adapter stays fixed for an entire training or evaluation cell.

text-state. The state, goal, and search context are serialized as a compact relational representation. Object identities, predicate names, and all Search Memory contents (frontier, visited set, algorithm-specific state) appear as structured text.

visual-state. The state and goal are rendered as a pair of 128×128 pixel images, a rendered state image and a partial-goal constraint image. Object labels are not embedded in the rendered image. Search Memory contents and the grounded candidate list remain in the textual serialization, even in this adapter. This is the published frozen `visual-state` arm that the native arms below are compared against.

The figures show cached policy scenes, enlarged but never retouched. Panel (a) of Figure 1 uses the cached 128-pixel unlabelled scene of the initial ferry state that the policy received, with vector labels added beneath the image rather than drawn into it, and Figure 2 shows the same cached scene as a thumbnail. The trace figures in Appendix O enlarge cached 128-pixel unlabelled scenes of one blocksworld task by nearest-neighbour sampling, without re-rendering. Their corruption panels regenerate the blank and degraded images with the same corruption function that the observation-corruption suite applied at evaluation time (Appendix K). The choice-frontier validation filmstrips in Appendix Q.4.2 show label-free 256-pixel re-renders, used only after the same scenes at the policy’s 128-pixel resolution matched the cached inputs pixel for pixel. The held-out puzzle shows the original cached 128-pixel frames instead, because re-rendering its initial state changes tile glyphs. Figure scripts retain the exact image paths and checks.

multimodal-state. The observation combines the text-state serialization and the visual-state rendered images. The policy receives the relational text and the image pair jointly.

Native image-only arms. Two further adapters executed under the native-arms v2 contract. `visual-nomem-state` keeps only the static task interface and the grounded, unscored candidate menu as text, and carries state facts in the current rendered scene alone. `visual-seq-state` adds up to $K=8$ rendered history frames, each labelled with its absolute path depth. Neither receives textual Search Memory, accepted deltas, g/h /priority scalars, candidate scores, or membership flags.

Text-scaffold caveat. In all three declared adapters, including visual-state, search bookkeeping travels as text. Rendered images carry the planning state and goal visuals, but the frontier, visited set, novelty counters, and candidate list are always serialized as structured text. The observation-corruption suite is consistent with this (Table 9). Copyability-preserving text shuffling drops success by up to -0.944 (text) and -0.722 (multimodal), while visual corruption costs at most -0.111 with intervals that include zero.

The native arms narrow this caveat to the candidate menu without removing it. With the textual bookkeeping and scalars removed, both arms solve 18/18 clean tasks with zero invalid operations, and blanking or degrading every image of the seq arm gives a corrupted-minus-clean contrast of 0.000 [0.000, 0.000] (Appendix M). As a post-hoc interpretation, the grounded unscored menu is the remaining text channel that could carry the signal. Menu manipulation shows that the policies

do not reject schema-valid inapplicable menu entries (pick-rate at chance), and on the additive cells submission-order invariance makes any listed candidate a correct emission (Appendix L). The caveat therefore still holds for every adapter that presents the menu, and no executed adapter isolates an image-only decision channel.

Information-loss boundaries. Matching the adapter between training and evaluation ensures identical observation format, not information parity, so the available evidence does not establish semantic parity across adapters or an intrinsic modality effect. The 128-pixel unlabelled rendered views can lose symbolic identities, and static-context and goal fields of the observation contain text in every adapter. The loss also differs by modality. The P3 name-compression family in the generalization-v2 suite is information-lossy, and family-wide (24 eligible P3 variants), 24/24 variants are text-lossy, 18/24 are multimodal-lossy, and 0/24 are visual-lossy.

E DATA AND SPLIT PROCEDURES

This appendix describes the corpora, splits, and evaluation manifests behind the reported panels, for readers checking that no evaluation task leaked into training.

Every training and evaluation split is by whole task instance, so no state, decision, or target from an evaluation task appears in any training row. Every released corpus also passed no-overlap, no-conflict, live/training-parity, and token-budget audits before use.

BFS pilot corpus. The BFS pilot corpus is split by semantic task identity, and its v8 evaluation panel draws only from the development side, so no panel task contributes training rows. The v6 text-state process corpus holds 25,109 process rows split into 12,994 train and 12,115 dev rows, drawn from 90 replay-verified trusted traces covering 90 distinct tasks, with zero cross-split task overlap. The v8 evaluation panel selects its 15 tasks from the development set. Corpus observations expose the canonical task context, exact grounded successor candidates, and full visited membership. Release audits recorded zero semantic, input, and input-target overlap, zero conflicting identical-input labels, zero teacher-operation rejections, and zero live/corpus mismatches, all within an 8,192-token input cap and a 384-token output allowance.

BFWS corpus. The BFWS corpus covers the frozen 105-task development panel, and its audit counters recorded zero overlap, mismatch, rejection, or final-evaluation-access violations. It holds 69,019 replay-verified teacher decisions. Its release contains 69,019 process records, 67,215 operational records, and a process projection of 47,780 train and 21,239 dev rows. Each step’s observation exposes candidate duplicate membership, novelty values, goal-count priorities, insertion positions, and enqueue facts.

Enumeration training recipe. The enumeration-contract recipes differ in size across interventions, so they describe different development-stage interventions rather than a matched-size comparison. Each expanded-baseline process-SFT cell trains a fresh LoRA adapter for one epoch on 512 replay-verified records, whereas the separate BFWS gate checkpoint trains for two epochs on 47,780 records. The pinned Qwen3-VL-8B-Instruct backbone revision is 0c351dd0, and the program cap is 336 GPU-hours (Table 26).

Evaluation manifests. Every evaluation below is development-stage, and the one manifest reserved for final evaluation has never been accessed. The BFS pilot uses a frozen 15-task outcome-blind development panel. The BFWS structural gate froze a 105-task development panel and, separately, the manifest of the frozen 45-task final evaluation, which no model prediction, training decision, or analysis references. The expanded baseline evaluates 24 tasks from 12 domains under 4 algorithms, 3 modalities, and 4 arms (1,152 logical bindings, 576 model episodes, all independently replayed). The curriculum stage evaluates 27 unseen tasks. The generalization-v2 protocol admits 25 derived tasks from 93 eligible semantics-preserving variants (uniform $k=5$ cheapest-prefix per family, membership frozen before execution).

F STATISTICS

This appendix states the interval, test, margin, and tiny-strata conventions behind every reported contrast, for readers judging what an interval licenses.

Default contrast procedure. Main-program intervals are descriptive bounds, not evidence of broad superiority, because they come from one training seed and panels of 24 or fewer instances. Contrasts are paired by whole task instance, and paired differences use a bootstrap with 10,000 resamples at seed 1729 and 95% percentile intervals.

The BFWS gate’s reported lower bound is the 0.025 quantile, i.e., the lower endpoint of a two-sided 95% percentile interval. The gate follows the default procedure, with confidence level 0.95 and 10,000 paired whole-instance resamples (bootstrap seed 1729).

Tiny-strata rule. No intervals are computed over strata with fewer than 8 episodes, so such strata are reported as descriptive only. They are listed explicitly and never silently pooled or dropped.

Transfer tests. No transfer comparison survives multiple-testing correction, and this non-significance is not evidence of equivalence. Transfer comparisons use McNemar exact tests against the per-example base predictions, with Holm correction across all 36 adapter-by-benchmark comparisons, and the minimum adjusted p in the executed transfer branch is 0.8156.

Follow-up bootstrap conventions. The follow-up windows fix their own bootstrap seeds, separate from the main program’s seed 1729 default. The native-arms contrasts and the choice-frontier contrasts use a paired whole-problem bootstrap at seed 61813 with 10,000 resamples and 95% percentile intervals, and the second-backbone v3 contrasts (stress test R3) use seed 1729 with 10,000 resamples. The comparator-zoo metrics use a task-cluster bootstrap that draws nine tasks with replacement at seed 133 for 10,000 draws, keeping both additive cells and all random seeds inside their task cluster.

Materiality and non-equivalence. A follow-up contrast is material when its 95% interval excludes zero, and equivalence can be read only against a pre-registered margin. The follow-up protocols through the comparator zoo pre-registered no equivalence margin, so a non-material contrast there supports neither superiority nor equivalence. The later choice-frontier protocols pre-register an equivalence margin of ± 0.05 for D (adapter minus random-valid M1) and for the DAgger change Δ , and none for the separation S from an exact- ϵ rung, so a non-separated S is not equivalence to the rung. Wherever the text reports a non-material contrast or a comparison that fails Holm correction, an adjacent sentence states that non-significance is not equivalence. The exact sign-flip p value is likewise descriptive. It enumerates all 2^n sign vectors of the n per-task differences, is two-sided, and tests a zero difference, so it never supports equivalence.

Tiny strata in the follow-ups. The fewer-than-8 rule also governs discriminative subsets. The choice-frontier learned-minus-random-valid contrast is computed over 18 cells, but it rests entirely on storage, the only panel task where random-valid’s seed frequency lies strictly between 0 and 1 (0.2). A one-task discriminative subset falls under the rule, so that contrast is descriptive. The zoo’s M5 subset, defined from controls only, holds the same single task and is reported descriptively.

Zoo dominance tests. A positive learned-versus-comparator dominance claim in the comparator zoo must survive Holm step-down correction over fifteen tests (five comparators by three metrics), applied to paired task-cluster bootstrap one-sided p values. Undefined or uninformative co-solved contrasts prohibit a positive dominance claim and never count as wins. All zoo metrics are exploratory, and those computed on the stored choice-frontier episodes are post-hoc.

Seed-variance addendum. A prospective training-seed replication measured variance for exactly two cells, and its cross-seed aggregation is descriptive ($n = 3$ seeds). Seeds 17, 29, and 71 were frozen before any seed-29 or seed-71 artifact existed, and all 534 new episodes were independently replayed.

Table 3: **Process SFT beats the base and trails random-valid at all three seeds (descriptive).** Cell A, greedy $\times$ multimodal. Successes and paired differences compare process SFT with the base and random-valid.

Seed	Successes / 24	SFT – base	SFT – random-valid
17	21/24	+0.875	−0.125
29	21/24	+0.875	−0.125
71	22/24	+0.917	−0.083

Table 4: **Process SFT beats oracle-assisted random-valid on the BFWS development panel.** Development-stage success over 15 tasks for one fixed training seed, alongside the pretrained base and the exact reference. No final evaluation has been executed.

Condition	Success (15 tasks)
Pretrained base	0.0
Process SFT	0.933
Random-valid (oracle-assisted control)	0.467
Exact execution	1.0

In Cell A, process SFT beats the base and trails random-valid at every seed (Table 3). Random-valid solves every task of this additive cell by construction, because the runtime supplies the grounded candidates and orders the heap (Appendix P), so here it is a validity bound rather than a competitor. Cell A is the greedy $\times$ multimodal replication cell (`best_first_add_greedy` $\times$ multimodal-state) evaluated on the 24-problem panel, and each seed trained a fresh LoRA adapter with new initialization and dropout randomness on the identical frozen 512-record membership.

Cell set B replicates a null result at every seed. It compares DAGger iteration-1 with the exposure-matched continued-SFT control under BFS across all three modalities, and every arm at every seed has 0/72 unseen successes. The paired contrast (DAGger minus continued SFT) is therefore +0.000 [0.000, 0.000] at each seed, which is not evidence of equivalence.

This addendum supersedes the blanket single-training-seed boundary for these two cells only. Every other enumeration-contract cell in the program remains single-seed (seed 17) evidence.

G RESULTS DETAIL: PRIMARY-MATRIX TABLES

On the expanded baseline, at 512 training records per cell, process SFT succeeds on 125/288 entries, all in the two additive cells, where random-valid saturates by construction (Appendix P). It reaches no goal in the BFS and BFWS cells at this budget, and its seed-replication gain over the base also lies in an additive cell (Table 3). The BFWS positive comes from a separate fixed-seed run on the frozen text-state BFWS development panel with 47,780 training records, which beats random-valid with a paired bootstrap 95% lower bound of 0.2 (Table 4). The two tables keep what the model learned apart from what the controls achieve, and no measure is combined with another. Table 4 gives the BFWS development panel, and Table 5 gives the expanded 24-task baseline by algorithm and modality. Section 5.1 summarizes both.

H RESULTS DETAIL: PRIMARY-MATRIX PAIRED CONTRASTS

This appendix gives the paired whole-problem contrasts of the expanded baseline, for readers asking how the learned gain compares with each control.

Process SFT at 512 records gains over the base only in the additive cells (Table 6). Against the base it gains +0.833 under weighted A* and +0.903 under greedy best-first, and nothing under BFS or BFWS. Against random-valid it loses in every algorithm column and wins none of the 72 problem-modality units in any column. In the two additive columns random-valid ties exact execution by construction, because the runtime supplies the grounded candidates and orders the heap (Appendix P). There random-valid is an oracle-assisted validity bound rather than a competitor, and the gap to it equals the adapter’s failure rate. The development-panel BFWS positive, trained on 47,780 records, does not reappear on this suite at 512 records. These contrasts are tiny-subgroup

Table 5: **At 512 records, learned success appears in the additive cells, where random-valid saturates by construction.** Descriptive enumeration-contract baseline by algorithm and modality. Successes, SFT invalid operations per decision, SFT expansions, and exact-reference decisions are separate columns. Random-valid (RV) and exact are oracle-assisted controls, not model abilities.

Algorithm	Modality	Base	SFT	RV	Exact	SFT inv/dec	SFT exp.	Exact dec.
BFS	text	0/24	0/24	15/24	24/24	24/66	15	1969
	visual	0/24	0/24	15/24	24/24	24/69	17	1969
	multimodal	0/24	0/24	15/24	24/24	24/63	15	1969
BFWS	text	0/24	0/24	17/24	24/24	24/24	0	974
	visual	0/24	0/24	17/24	24/24	24/24	0	974
	multimodal	0/24	0/24	17/24	24/24	24/24	0	974
Weighted A* ($w=3$)	text	0/24	20/24	24/24	24/24	4/609	179	694
	visual	0/24	18/24	24/24	24/24	6/499	143	694
	multimodal	0/24	22/24	24/24	24/24	2/666	196	694
Greedy best-first	text	0/24	22/24	24/24	24/24	2/614	179	673
	visual	0/24	22/24	24/24	24/24	2/643	189	673
	multimodal	0/24	21/24	24/24	24/24	3/617	179	673

Table 6: **At 512 records, process SFT gains over the base only in the additive cells, and random-valid bounds it in every column.** Descriptive paired success contrasts. Each cell gives the point difference and wins/losses/ties over problem-modality units.

Contrast	BFS	BFWS	Weighted A* ($w=3$)	Greedy
SFT – base	+0.000 (0/0/72)	+0.000 (0/0/72)	+0.833 (60/0/12)	+0.903 (65/0/7)
SFT – random-valid	−0.625 (0/45/27)	−0.708 (0/51/21)	−0.167 (0/12/60)	−0.097 (0/7/65)
SFT – exact	−1.000 (0/72/0)	−1.000 (0/72/0)	−0.167 (0/12/60)	−0.097 (0/7/65)
Random-valid – exact	−0.375 (0/27/45)	−0.292 (0/21/51)	+0.000 (0/0/72)	+0.000 (0/0/72)

descriptive bounds over 24 problems and one training seed, so they support no modality or algorithm superiority claim beyond the executed panel. The complete six-measure matrix for the frozen final protocol, including per-cell Algorithm Invariant compliance and charged-budget columns, belongs to the unexecuted frozen 45-task final evaluation.

I RESULTS DETAIL: SECONDARY PANELS

This appendix reports the secondary enumeration-contract panels behind the modality, robustness, and transfer findings (Sections 5.2, 5.3, and 5.4), together with the remaining enumeration-contract panels and the second backbone.

Modality. No executed enumeration-contract panel shows a modality effect. The per-modality rows of Table 5 have the same profile in all three adapters: zero learned success under BFS and BFWS, and learned success below the saturating controls in the additive cells. The earlier matched-modality study, a 3-problem panel on its own ledger that is never pooled with the expanded baseline, agrees. Its process-SFT success is 5/12 in text-state, 6/12 in visual-state, and 6/12 in multimodal-state, against 10/12 for random-valid in each modality, 0/12 for the base, and 12/12 for exact execution. A difference, had one appeared, could not be read as an intrinsic modality effect, because semantic parity across adapters is not established. The unlabelled 128px rendered views can lose symbolic identities, and the static and goal pages still contain text in every modality.

Generalization and robustness v2. The learned additive best-first adapters succeed on most semantics-preserving perturbations, and shifted initial states hurt most (Section 5.3). The reduced v2 protocol admitted a uniform cheapest-prefix sample of 5 variants per family over 5 perturbation families (25 of 93 eligible variants, membership frozen before execution). Learned adapters succeeded on 128/150 episodes, against 0/150 for the pretrained base, whose every episode is a one-call invalid termination (150 decisions total). By family, learned success was 29/30 on object-renaming, 27/30 on name-compression, 27/30 on render-restyle, 26/30 on scale-up, and 19/30 on shifted-init. The modality split was text 44/50, multimodal 43/50, and visual 41/50, and the algorithm split was

Table 7: **Dagger does not improve BFS over exposure-matched continued SFT.** Development-stage success and unseen invalid-operation rate for the iterated arms, the original process-SFT checkpoint, and both references.

Arm	Dev success /9	Unseen success /72	Unseen invalid rate
Dagger iteration 2	3/9	1/72	0.316
Continued SFT iteration 2	2/9	1/72	0.175
Original process SFT	1/9	0/72	0.364
Random-valid	6/9	45/72	0.000
Exact reference	9/9	72/72	0.000

greedy 67/75 and weighted A* 61/75. All 22 learned failures are single-invalid-operation terminations, 11 of them in shifted-init.

The controls saturate (random-valid 750/750, exact 150/150), so they remain bounds rather than a measure of learned ability. The lossy name-compression stratum is reported separately and never pooled. All 5 admitted variants are text-lossy, 3 multimodal-lossy, and 0 visual-lossy, and learned success on that stratum is text 9/10, visual 8/10, and multimodal 10/10. Family strata hold 5 problems each, so family-level rates are descriptive only. The full-scope v1 generalization branch was withdrawn after its terminal VALID_STOP receipt (closed 2026-09-21) and will not be executed.

Dagger. Dagger (Ross et al., 2011) does not improve BFS execution over an exposure-matched continued-SFT control, and both stay far below random-valid (Table 7). The comparison ran two DAGger iterations on 3 development and 24 unseen modality-task pairs. On the unseen panel, DAGger and continued SFT each achieve 1/72 invariant-valid successes, with one DAGger win, one loss, and 70 ties over whole problems. Random-valid reaches 45/72 and exact execution 72/72, while the original process-SFT checkpoint achieves 0/72. DAGger also has the higher unseen invalid-operation rate of the two iterated arms (0.316 versus 0.175). Final cumulative unique training corrections were text 201, visual 203, and multimodal 181, accumulated over 12 training cells of 512 records and 16 optimizer updates per cell-iteration.

Model-generated successors. The learned policy depends on the runtime’s trusted successors. When the model generates dynamic-state successors itself, goal success collapses from 45/45 to 1/45 on the evaluated panel, and the single success is the one multimodal unseen cell. The trusted arm made zero trusted-state substitutions. The generated successors fail mostly on content rather than format. The trained successor model is exactly correct on 65/109 retained raw predictions, with 93 schema-valid but only 66 effect-valid, and downstream failure kinds are dominated by exact state-identity and effect failures (27 effect, 16 schema, 1 static-context). Collection-time verification of the untrained exact-successor accepted 119/1,536 predictions (text 43, visual 52, multimodal 24). Coverage is partial by design. The frozen outcome-blind admission accepted a 12-task fallback panel after the full 24-task unseen panel was priced at 147.66 GPU-hours against the 64 GPU-hour branch cap, so full unseen coverage remains missing.

Curriculum by modality. Training order shows no material interaction with modality, although saturated controls leave little room for one to appear. Nine training cells crossed three modalities with staged, shuffled, and mixed ordering, all under greedy additive best-first and trained fresh from the same base on identical record sets, and were evaluated on 27 unseen whole problems. Successes were 24/24/25 (staged/shuffled/mixed) in text-state, 24/27/26 in visual-state, and 25/27/27 in multimodal-state. A sequential-SFT comparator reached 24/27, 25/27, and 24/27 in text, visual, and multimodal cells respectively. All four paired interaction intervals include zero: $-0.111 [-0.259, +0.037]$ and $-0.074 [-0.259, +0.074]$ for the visual and multimodal staged-minus-shuffled contrasts against text, and $-0.074 [-0.259, +0.111]$ and $-0.037 [-0.185, +0.111]$ for the corresponding mixed-minus-shuffled contrasts. The base stayed at 0/27, and random-valid and exact execution saturated at 27/27 in every modality, so ceiling and floor effects bound the observable differences.

Transfer. The BFWS-trace adapters show the largest transfer gain, raising FOLIO accuracy from 0.600 to 0.645–0.660 under all three observations, a consistent gain that is not statistically significant. All 12 verified planning adapters and the base were evaluated zero-shot on frozen subsets of

Table 8: **The BFWS-trace adapters show the largest FOLIO gain over the base, and no comparison survives Holm correction.** Descriptive accuracy of the 512-record expanded-baseline adapters on frozen FOLIO, GSM8K, and HumanEval subsets. Rows name each adapter’s trained algorithm with its internal cell identifier, and t/v/m denote its training observation.

Adapter	Modality	FOLIO	GSM8K	HumanEval
Base	—	0.600	0.955	0.780
BFS (bfs)	t	0.600	0.950	0.793
	v	0.605	0.945	0.799
	m	0.605	0.965	0.780
BFWS (iw)	t	0.660	0.960	0.768
	v	0.645	0.965	0.762
	m	0.645	0.960	0.756
Weighted A* (astar_w3)	t	0.590	0.950	0.817
	v	0.575	0.965	0.799
	m	0.590	0.945	0.774
Greedy (astar_greedy)	t	0.590	0.950	0.799
	v	0.590	0.955	0.811
	m	0.590	0.950	0.793

FOLIO (Han et al., 2022) (200 items), GSM8K (Cobbe et al., 2021) (200), and HumanEval (Chen et al., 2021) (164), with per-adapter accuracies in Table 8. These are the 512-record adapters of the expanded baseline, so the BFWS adapters that raise FOLIO solve no BFWS planning task at that budget (Table 5). Every adapter stays within 0.945 to 0.965 on GSM8K (base 0.955) and within 0.756 to 0.817 on HumanEval (base 0.780). McNemar exact comparisons (McNemar, 1947) of each adapter against the per-example base predictions, Holm-corrected across all 36 comparisons (Holm, 1979), leave none significant. The minimum adjusted p is 0.8156, and the smallest raw p is 0.0227 (FOLIO with the text BFWS adapter, at +12 examples). This null does not establish equivalence. A leakage screen over the frozen 8-gram window finds 7/564 benchmark items sharing only trivial numeric 8-grams with the planning corpus and zero content overlap.

Second backbone. A second backbone, InternVL3.5-8B (Wang et al., 2025), reproduces the enumeration-contract pattern across two reduced-scope cells. Each cell used a frozen 12-task key-cell panel selected by lowest reference decision count before any model outcome, with one seed-17 training run per modality cell. Under BFS (protocol v2) neither arm reached a goal. The pre-trained base scored 0/36 at 1 decision per episode (36 total), and process SFT scored 0/36 at 1 to 4 decisions (59 total), while the reused BFS controls scored random-valid 54/72 (18/24 per modality, oracle-assisted) and exact 72/72. Under greedy additive best-first (protocol v3, stress test R3), the same backbone, recipe, and panel shape give high learned success, with random-valid at its validity ceiling. Process SFT succeeds on 11/12 text-state, 10/12 visual-state, and 10/12 multimodal-state problems (31/36), while the base scores 0/36 with one invalid call per episode. Process-SFT invalid-operation rates are 0.00546, 0.01220, and 0.01258, and decision usage is 183, 164, and 159 of a 390-call allowance (text, visual, multimodal). Random-valid and exact reference both score 24/24 per modality. These control counts hold two entries per panel task, and both entries of a task come from its one reused, hash-pinned greedy baseline control episode per condition and modality. Paired whole-problem contrasts (seed 1729, 10,000 resamples) give SFT minus base +0.917 [+0.750, +1.000] in text-state and +0.833 [+0.583, +1.000] in both visual-state and multimodal-state. SFT minus random-valid is −0.083 [−0.250, +0.000] in text-state, and −0.167 [−0.417, +0.000] in both visual-state and multimodal-state. Coverage is complete, with 72/72 model episodes and 72/72 comparators replayed. The two InternVL cells reproduce the pattern seen on Qwen3-VL at the 512-record budget. The greedy additive variant is learned (31/36 against 0/36 for the base), with random-valid at 24/24 per modality because in additive cells it saturates by construction (Appendix P), while BFS is not learned. The earlier total null on this backbone is therefore consistent with an algorithm difference, the same one seen on Qwen3-VL, rather than a backbone null. The cross-backbone contrast against the pinned Qwen3-VL process-SFT rate is descriptive rather than paired, because the InternVL panel is the key-cell subset while the Qwen rate is the pinned full-panel aggregate. It is +0.000 [−0.167, +0.083] in text-state, −0.083 [−0.333, +0.083] in visual-state, and −0.042 [−0.292, +0.125] in multimodal-state. InternVL3.5-

8B shares the Qwen3-8B LLM family (Yang et al., 2025), so the comparison differs in the vision tower, connector, image tokenization, and multimodal recipe, and it is reported as an architecture-difference limitation. The full-scope v1 second-backbone branch was withdrawn after its terminal VALID_STOP receipt (closed 2026-09-21) and will not be executed, and its gate is retained as history.

J RESULTS DETAIL: FAILURE-MECHANISM CALIBRATION

This appendix localizes where each arm class fails, for readers asking why learned success is absent under BFS and BFWS at the 512-record budget.

Each arm class fails in its own way. The base mostly cannot produce the output format, learned arms mostly emit well-formed operations that break the search invariant early, and the oracle control runs out of budget late. These profiles come from a CPU-only calibration, frozen before any outcome was computed, that mined 2,910 unique replay-verified episodes across the five executed model branches and classified all 905 failures with zero unclassified. The analysis is descriptive. The base policy fails in 447 of 447 episodes, always on the first decision. Of these, 379 emit a malformed output that never reaches the operation checker, and 67 issue rejected BFS frontier operations. Learned arms fail in 407 of 906 episodes. Their dominant failure kind is `other_invariant_violation` with 229 occurrences (56.3% of learned failures), namely parsed, source-correct, applicable operations that the runtime rejects because they violate the declared search invariant. The remaining learned failures are 72 malformed outputs, 56 inapplicable grounded actions, 27 effect failures, 16 schema failures, and smaller kinds. First failures cluster early, at decision indices 0, 1, and 2 with counts 137, 88, and 46. The oracle control fails 51 times, all late and all by expansion-budget exhaustion (buckets 5–8, 9–16, 17–32, and 33+ hold 9, 9, 3, and 30). Table 2 gives the rejection taxonomy.

The failures concentrate where the primary matrix shows no learned success (Table 5). Every base-line process-SFT episode under BFS or BFWS fails (144/144), and the DAgger-iteration BFS arms fail 155/162, while the learned additive best-first families fail 56 of 546 episodes (10.3%). Process supervision thus leaves mainly an invariant problem rather than a format problem, since most learned failures are rejected invariant violations and malformed outputs are a minority. Read with the horizon profile, the learned policy fails early on invariant grounds, whereas the oracle control, which always emits valid operations, fails only when its expansion budget runs out late in the episode. Like the rest of the calibration, this reading is descriptive.

At 512 records the two non-additive algorithms fail in different ways, which the pooled profile hides. Under BFS the adapters emit well-formed search steps whose queue bookkeeping is wrong. Of the 72 main-grid BFS failures, 68 are successor operations with a wrong retire or position field and 4 retire the frontier prematurely. Under BFWS every one of the 72 main-grid failures is a malformed output at decision 0, so the adapter never emits a well-formed first operation. The BFS adapters have therefore learned the output format but not the queue discipline, whereas the BFWS adapters have not yet learned the format at this budget.

K RESULTS DETAIL: OBSERVATION-CORRUPTION CHANNEL TESTS

This appendix reports the observation-corruption tests, which show which observation channel the trained enumeration-contract controller relies on.

Corrupting the text collapses the trained enumeration-contract controller’s success, while corrupting the images barely moves it. The suite evaluated frozen checkpoints, with no retraining, on a prospectively frozen set of $k=9$ admitted tasks under four deterministic corruption families applied at observation time. Clean learned success on these tasks is 0.917 (text-state 0.944, visual-state 0.889, multimodal-state 0.889, with 9 tasks $\times$ 2 algorithms per cell). Copyability-preserving text shuffling lowers success by up to -0.944 , and the multimodal arm retains at most 0.167 success under shuffled text. Masking instead destroys the output contract. Blanking or degrading the rendered state costs at most -0.111 , and every visual interval includes zero, so visual-channel corruption is not material (Table 9). The multimodal channel-isolation contrast, paired by task and algorithm ($n=18$), measures the gap directly. Text-channel corruption costs -0.806 against -0.111 for visual-channel corruption, giving text minus visual -0.694 with 95% interval $[-0.889, -0.472]$, which is

Table 9: **Text corruption collapses the success of frozen checkpoints and visual corruption does not.** Paired differences compare corrupted with clean runs. ✓ marks a material interval-based verdict and = a non-material verdict.

Corruption	Arm	Corrupted	Clean	Δ	95% interval	Material
Text-masked	text	0.000	0.944	-0.944	$[-1.000, -0.833]$	✓
Text-masked	multimodal	0.000	0.889	-0.889	$[-1.000, -0.722]$	✓
Text-shuffled	text	0.000	0.944	-0.944	$[-1.000, -0.833]$	✓
Text-shuffled	multimodal	0.167	0.889	-0.722	$[-0.889, -0.500]$	✓
Visual-blank	visual	0.833	0.889	-0.056	$[-0.167, +0.000]$	=
Visual-blank	multimodal	0.778	0.889	-0.111	$[-0.278, +0.000]$	=
Visual-degraded	visual	0.833	0.889	-0.056	$[-0.167, +0.000]$	=
Visual-degraded	multimodal	0.778	0.889	-0.111	$[-0.278, +0.000]$	=

material under the frozen rule. This contrast pools the output-contract-destruction (masked) regime. The controls behave as the contract predicts. The corrupted base stays at floor (0/144, all first-decision invalid), exact execution remains at 18/18, and random-valid succeeds 72/72 on the new frozen seeds, so per the saturation rule no structural-advantage claim is made.

Masking and shuffling the text fail for different reasons, which the success deltas pool together. A decomposition of the 72 published text-corruption episodes (stress test R2) separates the two regimes. Under text masking, every episode of the text-state and multimodal-state cells is all-invalid (18/18 in each, 18 invalid operations over 18 decisions). Each emitted operation fails the schema, so the policy never enters the search. This is output-contract destruction, not information loss. Under copyability-preserving shuffling, by contrast, valid operations keep flowing while the search fails. In the text-state cell 16/18 episodes are partial-valid, 2/18 are all-invalid, and none reaches the goal (18 invalid operations over 49 decisions). In the multimodal-state cell 15/18 are partial-valid, none is all-invalid, and 3/18 reach the goal (15 invalid operations over 105 decisions). This partial-valid pattern is the information-loss signature. A new cross cell shows the same format failure on an image-state arm. Text masking applied to the frozen visual-state arm scores 0/18 with every episode all-invalid, so masking the text scaffold destroys the output contract even when images are the nominal state channel.

The language-channel claim survives only in its shuffled form. The masked catastrophe in Table 9 is mostly a loss of the copyable output format rather than a loss of state information. Shuffling the text leaves operations copyable and still stops the search, so under the frozen observation contract the textual serialization carries decision-relevant information for these adapters. Removing or degrading the visual channel barely moves success, including on the visual-state arm whose only rendered-semantics channel was corrupted. Appendices L and M probe the grounded candidate menu directly. This is a channel finding about the trained controller under the declared adapters, not evidence about intrinsic modality value.

L RESULTS DETAIL: MENU-MANIPULATION RECEIPTS

This appendix reports the menu-manipulation receipts, for readers asking whether the trained enumeration-contract policy reads its menu’s content or only its surface.

Menu manipulation (stress test R1) asks whether the trained policy responds to the content of its grounded menu. It runs on the frozen image-only adapters of both native arms (Appendix M) with no retraining. Menu permutation reorders the candidate menu, and distractor injection adds $k=3$ schema-valid but inapplicable actions to it. Table 10 reports the 72 episodes, all independently replayed within the 170/170 stress-evaluation episodes.

The distractor receipt shows that the policy does not reject schema-valid inapplicable menu entries, but it cannot tell content reading from position habits. On the frozen native adapters (stress test R1, 72 replayed episodes), injecting $k=3$ schema-valid but inapplicable actions collapses success to 0/36, with 36/60 invalid operations and a distractor pick-rate of 0.6 against a uniform-picking expectation of 0.592 to 0.625. Every episode ends on a distractor pick, 20/36 on the first decision. The pick-rate is therefore at chance, and these out-of-distribution distractors never appeared in training

Table 10: **Reordering the menu changes nothing, while injected inapplicable actions end every episode.** Menu manipulation on the frozen native arms. Invalid operations count emitted operations, and distractor picks count decisions with injected distractors.

Family	Success	Invalid ops	Distractor picks
Menu permutation	36/36	0/512	n/a
Distractor injection	0/36	36/60	36/60 (0.6)

menus. Content-reading and position habits both predict a pick-rate near 0.6, so this receipt does not separate them, and the discriminating fixed-versus-random-position test remains unrun. Menu permutation leaves 36/36 successes and 0/512 invalid operations, which is uninformative about order under the submission-order invariance of Appendix P.

M RESULTS DETAIL: IMAGE-ONLY NATIVE ARMS

This appendix reports the image-only native arms, which test whether removing the textual scaffold changes the enumeration-contract results.

The native-arms follow-up (contract v2) removes every textual search aid except the grounded candidate menu. In its two image-only arms, `visual-nomem-state` (nomem) and `visual-seq-state` (seq), state facts arrive only as rendered images, and seq adds a window of $K=8$ history frames. Neither arm receives textual search memory, g/h /priority scalars, candidate scores, or membership flags, but the grounded, unscored candidate menu is still presented. Both arms passed a pre-registered post-training smoke gate with 62/62 schema-valid grounded calls against a threshold of 0.5. That gate comes from the v2 amendment, frozen before any trained-arm outcome, which replaced the v1 admission gate on pretrained-base format compliance (retained as history). Four adapters (two arms by two additive best-first algorithms) used the frozen recipe with 512 records, 16 updates, and seed 17. All 216/216 new bindings were recorded and independently replayed, together with 72/72 reused replay-verified comparators. The window spent 3.2086 of 12 GPU-hours.

Without textual bookkeeping both arms still solve every clean task, but a saturated control leaves no room to call this an advantage. Both arms nominally succeed on 18/18 with zero invalid operations, against 16/18 for the published frozen `visual-state` arm. The pretrained base scores 0/36 under both new contracts, and the exact reference scores 18/18. Random-valid is at ceiling on its five frozen seeds, so under the saturation rule no structural-advantage claim is made. Paired whole-problem contrasts ($n=18$, bootstrap seed 61813, 10,000 resamples, 95%) give nomem minus `visual-state` +0.111 [0.000, +0.278] and seq minus `visual-state` +0.111 [0.000, +0.278]. Neither is material, and neither establishes equivalence. The history window changes nothing on these tasks (seq minus nomem 0.000 [0.000, 0.000]). Learned minus pretrained base is +1.000 [+1.000, +1.000] for each arm, which is material.

The pre-registered corruption payoff is a null, so the images-only hypothesis fails on this panel. Under seq the images are the only state and history channel, yet blanking every attached image gives a seq corrupted-minus-clean contrast of 0.000 [0.000, 0.000], and degrading every image gives 0.000 [0.000, 0.000]. As a post-hoc interpretation, the grounded unscored candidate menu is the remaining channel that could carry the signal, since neither the rendered scenes nor the removed scalar scaffold is necessary once the menu is present. That reading has a limit. On these additive cells the runtime orders the heap itself, so any listed candidate is a correct emission (Appendix P), and the null is equally consistent with a policy that uses no state information.

The native arms reach ceiling on the two generalization strains where the full-scaffold adapters degraded, although this comparison is post-hoc and unpaired. The degraded-strain test (stress test R4) evaluated the same frozen native adapters on the shifted-init and scale-up families. The learned adapters succeed on 10/10 shifted-init and 10/10 scale-up episodes for both arms, and the pretrained base scores 0/20 on each strain (0/40 in total). The published full-scaffold rates on the same variants are 19/30 for shifted-init and 26/30 for scale-up (Appendix I). The two rates come from different episode sets, with 10 visual-only episodes per arm against 30 full-scaffold episodes pooled over three modalities. Read with that caveat, the result is consistent with the reduced contract removing

scalar operands whose emission the full-scaffold adapters failed. It does not measure whether that text carried signal, since any listed candidate succeeds on these cells.

N RESULTS DETAIL: SEED REPLICATION AND FOLLOW-UP LIMITS

This appendix shows that the enumeration-contract seed replication reproduces the seed-17 verdicts, and it lists the development-stage limits of every follow-up window.

Both replicated enumeration-contract cells give the same verdict at all three training seeds. The prospective replication froze seeds 17, 29, and 71 before launch, independently replayed 534 new episodes, and executed on two cells (Table 3). In the greedy $\times$ multimodal replication cell (greedy best-first $\times$ multimodal-state, 24-problem unseen panel), per-seed success is 21/24, 21/24, and 22/24 (mean 0.889, range [0.875, 0.917]). The contrast against the pretrained base is positive at every seed (+0.875/+0.875/+0.917, no sign flip), and the contrast against random-valid stays negative ($-0.125/-0.125/-0.083$). On the DAGger iteration-1 cell set (BFS $\times$ three modalities, DAGger and continued-SFT arms), every arm and seed scores 0/72 unseen, with paired contrast +0.000 [+0.000, +0.000] at each seed, and the published iteration-2 verdict (1/72 vs 1/72, seed 17) is unchanged.

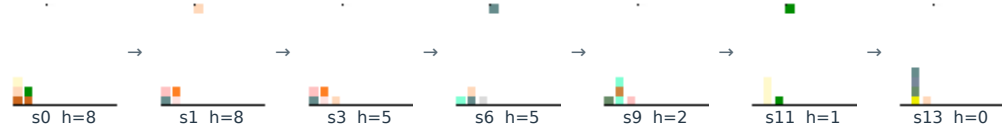
The replication covers only part of the enumeration matrix. Cell A covers one of the three modalities of the greedy additive cell. The remaining two modalities and all other enumeration-contract cells use seed 17 only, and cross-seed aggregation is descriptive ($n=3$). The image-only-arm proposal has since executed under the native-arms v2 amendment (Appendix M), and its v1 gate remains as history.

Every follow-up window is development-stage, and none accessed the frozen 45-task final evaluation. The native-arms window evaluates an 18-unit clean panel with one training seed per cell. The R1-R4 follow-up window reuses the frozen native-arms adapters for R1, R2, and R4, and trains one seed-17 run per cell for R3. The choice-frontier window trains one seed per algorithm on a 9-task panel whose discriminative subset holds one task. The comparator zoo is a CPU-only exploratory analysis. The adapter has three training seeds on the validation panel, the held-out panel, and the unscreened panel, and the DAGger ablation trains one seed on half the planned corpus. The shared nine-task panel draws from 8 domains with 7 compact and 2 expanded variants.

Final primary evaluation prerequisites. No authorized protocol yet exists for the frozen 45-task final evaluation. Freezing one requires recorded decisions on the following prerequisites. (1) The condition-level data source and Search-Trace Segment construction with per-algorithm allocation. (2) Compute parity and checkpoint-selection rules. (3) The episode-budget unit, the whole-instance split procedure, and the fixed Search Memory capacity. (4) The estimand, the uncertainty interval or test, the success threshold, the invariant-compliance rule, and the validity and invalid-operation denominators for every cell. Until these choices are recorded, the final evaluation protocol cannot be frozen and no final comparative efficacy finding exists.

O ILLUSTRATIVE SEARCH EPISODES

This appendix shows three enumeration-contract episodes whose scenes come from one blocksworld task. Figure 3 follows a solved greedy best-first episode decision by decision. Figure 4 contrasts how the 512-record BFS and BFWS adapters fail: the BFS adapter emits well-formed steps with wrong queue fields, whereas the BFWS adapter fails on output format at its first decision (Appendix J). Figure 5 shows the same task under text and image corruption (Appendix K). The episodes are illustrations, selected without a pre-registered rule, and they do not estimate a task-level effect.

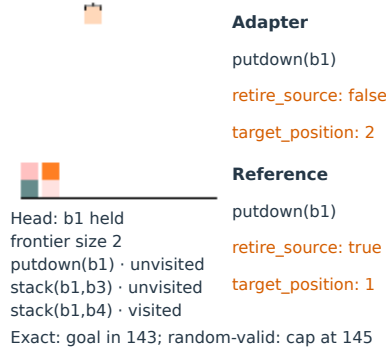
A trained greedy best-first search traceExpanded states (cached policy views; h_{add} alongside)**One expansion: s6, $h_{add}=5$** **Runtime heap expands s9 ($h_{add}=2$) next****Adapter emits (verbatim)****Textual successor rows**

action	g	h	prio	state	
putdown(b4)	4	4	4	s7	<code>{"action":{"args":["b4"]}, "name":"putdown"}, {"source_state_id":"s6"}</code>
stack(b4,b1)	4	4	4	s8	<code>{"action":{"args":["b4","b1"], "name":"stack"}, {"source_state_id":"s6"}</code>
stack(b4,b2)*	4	5	5	s3	<code>{"action":{"args":["b4","b2"], "name":"stack"}, {"source_state_id":"s6"}</code>
stack(b4,b3)	4	2	2	s9	<code>{"action":{"args":["b4","b3"], "name":"stack"}, {"source_state_id":"s6"}</code>

* closed/pruned by runtime

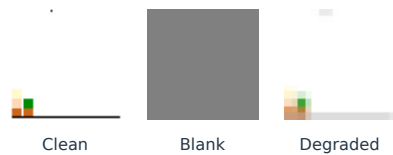
s6 · catalog state 8

Figure 3: **A trained greedy best-first adapter reproduces the reference search on a five-block task.** Top: the six states the runtime expanded, with their h_{add} , and the goal. Bottom: one expansion as the visual-observation adapter sees it, a rendered scene plus textual candidate rows with runtime-computed values, and the four operations it emits. Illustrative, not an efficacy estimate.

Why the 512-record BFS and BFWS adapters fail**BFS · queue fields****BFWS · operation schema**

Candidate eval.frontier: {true, 0}
 Reference: unstack(b1,b4); visit/evaluate
 source '\$'; frontier_intent {true, 0}
 Text: missing frontier_intent, visit_target
 invented: frontier_exhausted, exact_bfws_*
 Visual: frontier_target_state_id,
 retire_source_state (invented keys)
 Multimodal: invented-key runaway
 {"canonical_rationale":"exact_bfws_goa...
 1,712 characters; rejected at decision 1

Figure 4: **At 512 records, BFS fails on queue bookkeeping and BFWS on the operation format.** Left: the BFS adapter names a valid successor but misstates whether the head is retired and the queue position (differences in orange); all three observation types give this error. Right: the BFWS adapter names the reference action but writes a malformed operation.

Test-time corruption · same task and greedy checkpoints**Policy scene variants****Shuffled text**

Text: repeats unstack(b1,b4) at call 2
 Multimodal: repeats it at call 3
 Both rejected; previously submitted from s0

Blank / degraded images

Visual + multimodal: goal in 18 calls each

Figure 5: **The same greedy adapters fail when their text is shuffled and solve with blanked or degraded images.** With shuffled text, the text and multimodal adapters resubmit an operation they already submitted and are rejected at decisions 2 and 3. With blanked or degraded images, the visual and multimodal adapters solve the task in 18 decisions.

Table 11: **Random-valid saturates the additive cells, where the audit finds its decisions identical to the exact reference’s.** Descriptive development-stage successes out of 24 tasks per algorithm and observation at 512 training records, the same cells as Table 1. Base and controls do not vary with observation. The audit column counts decision-identical additive task–algorithm pairs on this runtime.

Algorithm	Base	Process SFT			Random-valid	Exact	Audit
		text	visual	multimodal			
BFS	0	0	0	0	15	24	—
BFWS	0	0	0	0	17	24	—
Weighted A* ($w=3$)	0	20	18	22	24	24	48/48
Greedy best-first	0	22	22	21	24	24	
Total (of 288)	0	125			240	288	

P IDENTITY AUDIT OF THE ENUMERATION CONTRACT

The identity audit compares random-valid with the exact reference decision by decision on the same tasks and seeds. Random-valid is an oracle-assisted control: at each decision it draws one of the runtime’s listed valid operations uniformly at random, with a fixed seed and no retry or repair. Where its decisions never diverge from the reference’s, its success measures operation validity, and it serves as a validity bound rather than as a competitor. On the enumeration contract the audit is a diagnostic, and on the choice-frontier contract it is a pre-registered pass condition (Appendix Q).

In the enumeration contract’s two additive cells, random-valid decides exactly as the exact reference on our runtime, on 48/48 task–algorithm pairs (Table 11). The executed serial rule causes this. Each expansion submits its complete candidate set, and equal-priority ties break in sorted candidate order, so the runtime’s priority heap fixes the order of expansion and the order of submission cannot change it. Random-valid therefore saturates the additive cells by construction, and success there measures whether the policy emits valid operations. The identity belongs to this controller rather than to additive best-first search in general (Appendix B), and the contract specification alone does not reveal it. In the BFS and BFWS cells random-valid’s decisions differ from the reference’s, and its success falls below the reference’s.

Success saturation alone could not have exposed the identity. Random-valid also saturated the 15-task BFS development panel, although its BFS decisions on the expanded baseline diverge from the reference’s. The audit therefore has to run on each interface’s executed code before random-valid success is read as a measure of choice (step 1 of Box 1).

Changing only the tie-break order breaks the identity, which confirms the serial rule, rather than the contract alone, as its cause. When equal-priority ties break in submission order, 19/48 pairs diverge, and on the remaining pairs no tie resolves differently under the two orders (Appendix P.1).

P.1 SUBMISSION-ORDER COUNTERFACTUAL

This subsection gives the protocol and the result of the counterfactual that locates the 48/48 identity in the executed heap-serial rule.

The counterfactual changes only the rule that assigns heap serials and re-runs the identity audit on the same 48 additive pairs (24 tasks $\times$ 2 algorithms, seed 17). The executed rule assigns serials at expansion start from the sorted candidate order. The counterfactual assigns them at application time in submission order. The protocol was committed before any output of this CPU-only follow-up, and the audit first re-validated 288 stored episode counts and 96 raw-output sequences with no mismatch.

The 48/48 identity is a property of the executed serial rule, not of the enumeration contract alone. Under the submission-order rule 19/48 pairs diverge (Table 12), while the executed rule stays identical as predicted. The 19/48 falls short of the pre-registered threshold of more than 24/48, so the stored verdict reads “PREDICTION_FAILED: see preregistered_prediction”. Where pairs diverge, the first divergence falls between decisions 2 and 50 (median 6), and both algorithms on one ferry task flip the goal outcome.

Table 12: **Heap-serial counterfactual identity audit.** Random-valid matches the exact reference on every additive pair only under the executed sorted-order rule. Each row compares random-valid and exact-reference choices on the same additive pairs.

Heap-serial rule	Identical	Divergent
Sorted candidate order (executed)	48	0
Submission order (counterfactual)	29	19

Q FRONTIER CHOICE FROM RENDERED SCENES

Under the enumeration contract the runtime’s priority heap decides which state to expand next, so the trained policy never chooses among frontier states (Appendix P). This appendix reports a separate development-stage study of that choice. Under the choice-frontier contract the policy selects, from rendered scenes alone, which runtime-owned frontier state to expand next, and the runtime performs the expansion. Random-valid no longer saturates this contract, so a success measure has to be shown to rank choices correctly before any policy is scored against it. Box 1 gives the four-step procedure we used, and the subsections follow its steps. They cover the contract (Appendix Q.1), the first measurement and the comparator zoo (Appendix Q.2), the validation ladder (Appendix Q.3), and the scaled adapter with its held-out panels (Appendix Q.4). Appendix R applies steps 1 and 2 to published interfaces.

Under this contract random-valid chooses uniformly among the offered frontier states. Like the exact reference, it is an oracle-assisted control whose score is a bound, not an ability. Results come from reused, freshly frozen, and new-seed panels, each kind further removed from our design decisions. The first measurement used the reused 9-task panel, low-cost tasks from 8 domains shared with other experiments, so its design could adapt to earlier results. The ladder test and the adapter use the validation panel, 12 fresh tasks from 7 domains frozen before evaluation. The held-out panel (P2 in figures and tables) applies the same rules to new seeds on which no adapter had been evaluated, and their range was extended once before any evaluation episode (Appendix Q.4.1). Held-out means this panel only, never the frozen 45-task final evaluation, which remains unexecuted.

The validation and held-out panels pass a random-valid screen that admits a task only if 1 to 9 of its 10 random-valid episodes reach the goal, which keeps tasks on which near-uniform choice sometimes succeeds. The unscreened panel (P2u), frozen with the held-out panel before evaluation, removes this selection by drawing kept candidates outside the held-out panel without the screen.

Each choice-frontier verdict follows a rule frozen before its outcome. The identity audit passes on any divergence over the frozen task–algorithm pairs, without re-runs. The ladder uses the pass rule in step 3 of Box 1. The adapter rule requires a positive difference from random-valid in M1, with a materiality margin of 0.05 and an equivalence margin of ± 0.05 . Later frozen protocols add three training seeds, two fresh panels, a zero-shot policy, and a second external audit.

Q.1 CONTRACT AND COMPARATOR SELECTORS

Under the choice-frontier contract the policy selects which runtime-owned frontier state to expand next, under a budget of twice the exact reference’s expansion count. The contract runs only the two additive best-first variants. Its observation is scene-only. The policy sees rendered scenes of the initial state and of each frontier state, plus goal pages, and opaque labels are the only candidate text. No observation modality is therefore compared under this contract, and its results are not comparable with enumeration-contract results (Table 28). The paragraphs below specify the menu, output, budget, corpus derivation, metrics, and comparator selectors.

Menu and observation. The choice-frontier observation withholds every score, position cue, and search counter, so a choice can rest only on the rendered scenes and the task pages. At each decision the runtime exposes the live frontier as an unscored menu of opaque labels $c_0, \dots, c_K$. The labels are assigned after a seeded Fisher–Yates permutation of the generation-serial member order, with a per-decision seed derived from master seed 51131 and a hash of the member state identifiers, so a label carries no positional information. The observation holds static task-context pages, the unlabelled 128px initial-state scene, one unlabelled 128px scene per frontier state in menu order, and the partial-goal pages. Its text payload is limited to the algorithm name, the label list, the

Box 1: Validate before score. Four steps admit random-valid as a choice control and validate the success measure before any policy is scored.

Four-step validation

Inputs. The executed interface (per-decision candidate set, ordering, tie-break rule), the exact reference, and seeded random-valid with no retry or repair.

1. Identity audit. Compare random-valid with the exact reference decision by decision on the same tasks and seeds, reporting the divergent fraction and first divergence. Read success as choice only where decisions diverge and random-valid's success falls below the reference's. Otherwise success measures validity, which saturation alone cannot reveal (Appendix P).

2. Budget check. If a zero-token control exists, follow the compute-matched-baseline principle (Li & Talwalkar, 2020). Count the budget in the units that the control spends, decisions or expansions, and report its success at the reference's count and at twice it. The check fails if the control saturates in its native unit but not in decisions (Appendix R).

3. Ladder. If the reference computes a priority order, freeze a fresh panel before evaluation and pre-register the order exact reference, ϵ 0.25, 0.50, 0.75, random-valid. Each exact- ϵ rung replaces a fraction ϵ of reference decisions with random-valid choices. Score M1, the area under the solve-versus-budget curve up to twice the reference's expansion count. The ladder passes only if all four adjacent paired task-clustered 95% lower bounds are positive along the ϵ axis. The rungs are privileged, so a pass validates the measurement, not a policy (Appendix Q.3).

4. Report. Give the policy's difference from random-valid with a paired task-clustered interval and a pre-registered margin (0.05 here), its ladder position, per-task signs, screened and unscreened panels, and training seeds. Non-separation from a rung is not equivalence (Appendix Q.4).

Where a step was at risk (one audited instance each). Complete-set submission with a priority heap and sorted tie-breaking failed step 1 (our enumeration contract's additive cells, 48/48). Per-step choice among valid actions passed step 1 (ScienceWorld). Model-scored candidates on a frontier that keeps every alternative, under a token budget, failed step 2 by construction (LLM-First Search).

If a step fails, change the contract (for example, let the policy choose the frontier state, or break ties in submission order) and audit again.

representation tag, the schema version, and the frozen legend, and a leak check fails closed on any forbidden key marker.

Output and validation. The output contract admits exactly one offered label per decision, and the first invalid emission ends the episode. The policy emits canonical JSON with the single key `expand_choice` and a label value. The runtime rejects malformed JSON, a wrong key set, a non-string label, a label not offered by the pending menu, and a label bound to a non-live member, and it records the rejection as `deterministic_invalid_operation`. An episode otherwise ends as `goal_reached`, `frontier_exhausted`, `decision_budget_exhausted`, or `expansion_budget_exhausted`.

Expansion and budget. The policy only chooses and the runtime does everything else, so one submitted choice is one decision. On acceptance the runtime removes the chosen member from the frontier and expands it atomically. Candidate generation, serial assignment, admission, duplicate detection, reopening, and expansion bookkeeping are inherited unchanged from the frozen additive best-first controller, which is not modified, and per-candidate admissions stay runtime-internal. Selecting a member that satisfies the goal solves the task without an expansion and without budget charge. The exact episode therefore takes its reference expansion count plus one decisions. Decision and expansion caps both equal $2 \times$ the reference expansion count.

Corpus derivation. Each training record teaches the exact reference's own choice, the label bound to the heap head. The training corpus was derived on CPU by replaying the frozen exact traces through the new contract and verified expansion-for-expansion against them (58 episodes bit-exact). Records are selected through the frozen matched-modalities membership, which indexes enumeration-contract decisions, and each such decision maps to the expansion whose candidate set it submitted into. Unique (task, expansion) pairs in first-occurrence order form the base set, and a

round-robin over the frozen task order tops the set up to 512 training records per algorithm. Diagnostics are the unique expansions of the frozen diagnostic identifiers, with no top-up.

Training ran only after five fail-closed gates passed. The gates check expansion-for-expansion agreement with the stored exact traces, a token budget of measured input plus 384 within 32,768, the text-leak check, image resolution, and identity with the frozen membership. The pinned preparation report records outcome PASS over 29 tasks, 1,046 records (512 training and 11 diagnostic per algorithm), 3,544 scenes, and 58 episodes re-derived bit-exact. The smoke gate accepted 40/40 model calls.

Training and replay. Both adapters trained to completion under the frozen recipe, and their episodes are independently replayed. Each of the two cells trains a LoRA adapter (rank 64, α 128) for 16 updates on its 512 records at seed 17 with the frozen expanded-program recipe. Both jobs completed all 16 steps and saved final adapters, and only the later post-save summary construction failed. Their reports were therefore recovered from the primary training-state artifacts with provenance marked, no retraining was needed, and the training audit passes on the recovered reports. At finalize, independent replay re-derives every menu binding, payload, and trusted transition from the stored events and checks pixel bindings against the retained scenes.

Comparator selectors. The comparator selectors replace only the choice. Each selector’s choice is mapped back to the menu binding, and every candidate still enters through the additive admission path, so frontier admission, duplicate detection, h_{add} computation, reopening, and serial ordering stay additive for `bfs-order` and novelty-first selection alike. Novelty-first selection uses positive PDDL state atoms as its novelty features, and its seen-feature sets advance only when a state is expanded, including the initial expansion. Each new episode is re-run against an independently constructed PDDL authority, which checks menus, decisions, admissions, and result dictionaries. This replay covers the model input, menu, actions, admissions, and budget, but selector episodes neither generate nor replay image bindings. The optimal costs C^* come from one offline shortest-plan computation per task with replay-verified plans, and the native four-algorithm reference costs are recorded separately from the zoo.

Metric computation. The protocol fixes each metric’s edge cases so that every arm is scored by the same rule. Under M1 an episode solves at multiplier m when its first goal selection occurs within $\lfloor mR_t \rfloor$ decisions with at most $\lfloor mR_t \rfloor$ preceding expansions, where R_t is the additive exact-reference expansion count. Unsolved $2\times$ runs count as failures at every m , the same prefix truncation applies to every arm, and task and algorithm weights are equal. M2 also has a censored form, in which failures are right-censored at the budget rather than assigned a fictitious expansion count. M3 counts transitions along the selected goal state’s trusted predecessor chain and is undefined, never imputed, for unsolved episodes. M4 measures agreement with the current additive heap head over decisions with at least two live choices, against a chance rate equal to the decision-weighted mean of $1/K$. M5 restricts to tasks where the mean $2\times$ random-valid solve indicator lies strictly between 0 and 1. The goal-at-selection rule fixes the reference’s own M1. Goal selection is the (R_t+1) -th decision, so the exact reference fails at $m = 1$ and scores M1 = 0.875 by construction. An arm that reaches the goal in fewer decisions can therefore exceed the reference, as the privileged hadd-greedy selector does on the validation, held-out, and unscreened panels (0.882, 0.928, and 0.906).

Q.2 FIRST MEASUREMENT AND COMPARATOR ZOO

This subsection documents the first measurement under the choice-frontier contract, on the reused 9-task panel, and the comparator zoo scored on the same panel. The contract passes the identity audit, because random-valid diverges from the exact reference on all 18 task–algorithm pairs of the panel. Both analyses predate the validated ladder. They show that the contract separates random-valid from the exact reference, but they cannot calibrate choice quality.

The first adapter (“Learned adapter” in Tables 13 and 14) was trained on a corpus that reproduces the exact reference and contains real choices. The corpus was derived on CPU by replaying the frozen exact traces through the new contract, and all 58 derived episodes matched their stored traces expansion for expansion. It covers 29 training tasks and holds 1,046 records (512 training plus 11 diagnostic per algorithm) with 3,544 scenes. Every corpus episode carries at least one decision

Table 13: **First choice-frontier measurement on the frozen panel.** Random-valid falls well below the exact reference, and the learned adapter emits no invalid operation. Cap denotes expansion-budget exhaustion and first-call an invalid first emission. These goal rates are not comparable with enumeration-contract performance.

Condition	Goal rate	Failures	Invalid ops
Exact reference	18/18	—	0
Learned adapter	7/18	11 cap	0
Pretrained base	0/18	18 first-call	18
Random-valid (5 frozen seeds)	32/90	58 cap	0

whose menu offers two or more states, with up to 315 such decisions in one episode and menus of up to 222 states. Two adapters, one per additive algorithm, were trained with the frozen recipe at seed 17 (16 updates on 512 records each). The smoke gate accepted 40/40 model calls, all 144/144 panel episodes were independently replayed with complete coverage, and the final audit returned CHOICE_SENSITIVE over those 144 replayed episodes. The window spent 1.069 of 12 GPU-hours (training 0.81, smoke 0.052, model evaluation 0.208), and corpus and controls ran on CPU.

The first measurement shows that the contract registers choice, but not that the first adapter chooses better than chance. Random-valid minus the exact reference is -0.644 $[-0.856, -0.422]$, material, so the two controls separate under this contract. The first adapter minus the pretrained base is $+0.389$ $[+0.167, +0.611]$, material, and with zero invalid operations this is a gain in contract fluency. The first adapter minus random-valid is $+0.033$ $[-0.033, +0.133]$, not material. Even that small difference is weaker evidence than its interval suggests. The 18 cells are 9 tasks measured under two algorithms as repeated measures, and the whole contrast rests on one task (storage), the only one where random-valid’s seed frequency lies strictly between 0 and 1 (it is 0.2). The frozen rule makes strata under eight cells descriptive only, so the contrast is descriptive. The evidence therefore supports no learning-to-plan claim for the first adapter, and no equivalence with random-valid, since no equivalence margin was pre-registered. All contrasts are paired percentile 95% bootstrap intervals over the frozen resampling cells (seed 61813, 10,000 resamples).

Clustering by task leaves both readings unchanged. Clustering is needed because the control expansion sequences coincide across the two algorithms on every panel task. With the same seed and resample count, the post-hoc task-clustered interval is $[0.000, +0.100]$ for the first adapter minus random-valid, which still reaches zero, and $[-0.889, -0.333]$ for random-valid minus the exact reference, which still excludes it.

Comparator zoo. The comparator zoo is an exploratory CPU-only comparison that places three deterministic rule selectors on the same measurement at the same $2 \times$ cap. `bfs-order` picks the minimum frontier g , then the earliest serial. Novelty-first selection picks the lowest width-1, then width-2, novelty over positive PDDL atoms. `worst-first` is a negative control. The comparison is exploratory because the metric set was pre-registered before comparator episodes, but the stored choice-frontier episodes enter post-hoc. It is also not information-matched. The selectors act on the additive frontier, not native BFS or BFWS, and read privileged scalars or PDDL atoms while the model sees only scenes, so no learned-versus-rule claim follows.

The zoo bounds the first adapter only against rules weaker than random-valid, so it cannot calibrate choice quality. On M1 the first adapter falls between `bfs-order` and random-valid and below novelty-first selection (Table 14, Figure 6), and no comparator lies between random-valid and the exact reference on this panel. The formal tests find no ordering either. Across the fifteen-test Holm family no comparison supports positive Pareto dominance for the first adapter, and three co-solved tests are undefined. Non-significance here is neither equivalence nor a general negative capability claim. The post-hoc label diagnostics and the one-task M5 subset below are descriptive only.

Zoo receipts. The zoo reuses the model’s episode conditions, so selectors and model face the same menus and budgets. Random-valid serves as the uniform-choice reference and the exact reference as the imitation target. Selector episodes replay the frozen model input, randomized menu, actions, admission, and budget, but they neither generate nor replay image view bindings. Because the selectors operate on the additive frontier, novelty-first selection does not reproduce BFWS pruning. Random-valid and the three selectors also ran at $3 \times$ and $4 \times$ caps, outside the primary metric. All

Table 14: **Exploratory comparator-zoo metrics at the shared cap.** No rule selector exceeds random-valid on M1, so no comparator lies between random-valid and the exact reference. M1 has task-clustered intervals, and M2 and M3 are solved-only. The rule selectors read privileged information unavailable to the model.

Arm	M1 [95% CI]	2× solved	3×	4×
Learned adapter	.1042 [.0278, .2014]	7/18	n/a	n/a
Pretrained base	0 [0, 0]	0/18	n/a	n/a
Random-valid	.1389 [.0250, .2722]	32/90	42/90	50/90
Exact reference	.8472 [.7917, .8750]	18/18	n/a	n/a
bfs-order	.0694 [0, .1528]	6/18	8/18	10/18
Novelty-first selection	.1250 [0, .2778]	6/18	8/18	12/18
worst-first	.0556 [0, .1389]	4/18	6/18	8/18

Arm	M2 ρ	M3 κ
Learned adapter	1.432	1.025
Pretrained base	undefined	undefined
Random-valid	1.407	1.000
Exact reference	1.000	1.000
bfs-order	1.613	1.000
Novelty-first selection	1.452	1.000
worst-first	1.549	1.050

Table 15: **Descriptive post-hoc M4 diagnostics for the first adapter.** Only the last-label excess has a task-clustered interval that excludes zero. The pooled column gives point differences from chance, and the task-clustered column adds 95% intervals.

Quantity	Pooled	Task-clustered [95% CI]
Heap-head agreement minus chance	−0.095	−0.069 [−0.161, +0.032]
Last-label rate minus chance	+0.276	+0.277 [+0.126, +0.402]

432/432 zoo bindings were observed and independently replayed, with 342 new CPU episodes, 90 original 2× random-valid controls reused, and none missing. The nine unit-cost optimal costs C^* come from one offline planner computation per task with replay-verified plans: storage 3; elevators, ferry, and 15puzzle-compact 4; depot, towers-of-hanoi, and visitall 5; blocksworld and 15puzzle-expanded 6.

Post-hoc diagnostics on the stored episodes find no heap-head agreement beyond chance and an excess of last-label choices that the data cannot attribute to a habit. The first adapter selects the reference’s heap-head state on 23/132 multi-option decisions (17.4%) against an on-policy chance rate of 26.9%, and it emits the last menu label on 72/132 (54.5%). These decisions cluster within 9 tasks. Under task clustering the agreement gap does not survive while the last-label excess does (Table 15). The post-hoc choice-frontier diagnostic records a first-label rate of 2.3% and a teacher-corpus last-label rate of 12.0% (chance 10.4%, 978 records). Menus are permuted per decision with a seed, and no last-label comparator was run, so these post-hoc rates do not establish a position habit.

M5 is too small to read beyond description. It restricts to tasks where random-valid solves at a 2× rate strictly between 0 and 1, and only one task qualifies (storage-compact-919000, random-valid rate .2). On it the first adapter’s M1 is .3125, against .075 for random-valid and .625 for the exact reference.

Clustering by task keeps the first adapter’s last-label excess but not its gap in heap-head agreement. The post-hoc analysis recomputes the M4 diagnostics over the first adapter’s 18 stored episodes on the earlier panel, resampling 9 task clusters (10,000 resamples, seed 133). Pooled over 132 multi-option decisions, heap-head agreement is 0.174 against a chance rate of 0.269, and the last-label rate is 0.545. The agreement gap’s interval includes zero while the last-label excess excludes it (Table 15). M4 therefore keeps its diagnostic, non-confirmatory status.

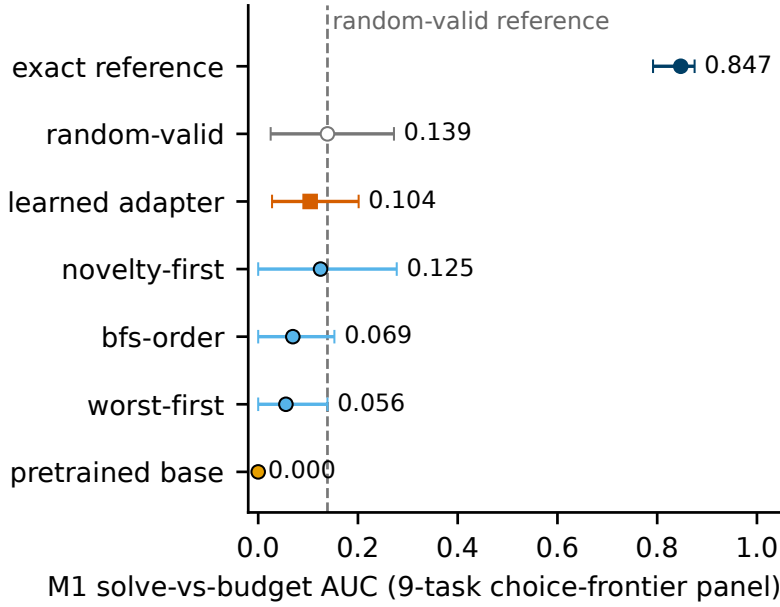


Figure 6: Only the exact reference stands apart on M1 (solve-versus-budget AUC at multipliers 1 to 2) on the frozen 9-task choice-frontier panel. The intervals of all non-exact arms overlap, and the Holm family licenses no ordering. Exact reference .847, random-valid uniform-choice reference .139, learned adapter .104 [.028, .201], novelty-first .125, bfs-order .069, worst-first .056, pretrained base 0. Under the enumeration contract, by contrast, the BFS/BFWS random-valid rows (15/24 and 17/24 versus exact 24/24) already registered choice.

Q.3 VALIDATION LADDER

On the choice-frontier contract, M1 passes its pre-registered validity test. On the fresh validation panel it orders the exact reference, the three exact- ϵ rungs, and random-valid as pre-registered, with every adjacent paired task-clustered 95% lower bound positive (Figures 7 and 8). The reference’s M1 of 0.875 is fixed by construction, because selecting the goal takes one decision beyond its expansion count (Appendix Q.1). The ladder also passes on the held-out panel, frozen before any evaluation episode, so the ordering holds on tasks from seeds no adapter had been evaluated on. It passes descriptively on the unscreened panel as well (Appendix Q.4.1).

A pass licenses one reading. Along the ladder’s ϵ axis, a higher M1 means choices closer to the reference’s (step 3 of Box 1). It does not validate M1 off this axis, because a privileged selector that reaches the goal in fewer decisions scores above the reference (Appendix Q.1). The rungs read the reference’s own priority order, which no policy sees, so the ladder validates the measurement, not a policy. Table 17 reports random-valid’s success at M1’s cap of twice the reference’s expansion count.

The rest of this subsection shows how the validation panel was frozen and gives every arm’s receipts on it.

The validation panel was fixed by rules committed in advance, the ladder protocol before task generation and the panel membership before evaluation. Of 240 generated candidates (12 snapshots, seeds 935000 to 935019), 127 met the cost filter ($8 \leq C^* \leq 20$), and 34 passed controls-only screening, which admits tasks on which random-valid is discriminative. The freeze rule sorts admitted tasks by domain and seed, keeps at most two per domain, and takes the first 12 (Table 16).

The ladder test passes on the validation panel because every adjacent gap has a positive lower bound. The pre-registered rule requires all four adjacent paired task-cluster lower bounds to be positive, with arms averaged over seeds within each task and algorithm, then over algorithms. The gaps widen toward the random end of the ladder: +0.082 [+0.051, +0.111] for the exact reference minus the 0.25 rung, +0.190 [+0.126, +0.252] for 0.25 minus 0.50, +0.256 [+0.184, +0.339] for 0.50 minus 0.75, and +0.326 [+0.233, +0.422] for 0.75 minus random-valid. All 696/696 CPU episodes and the 48 first-adapter model episodes were independently replayed.

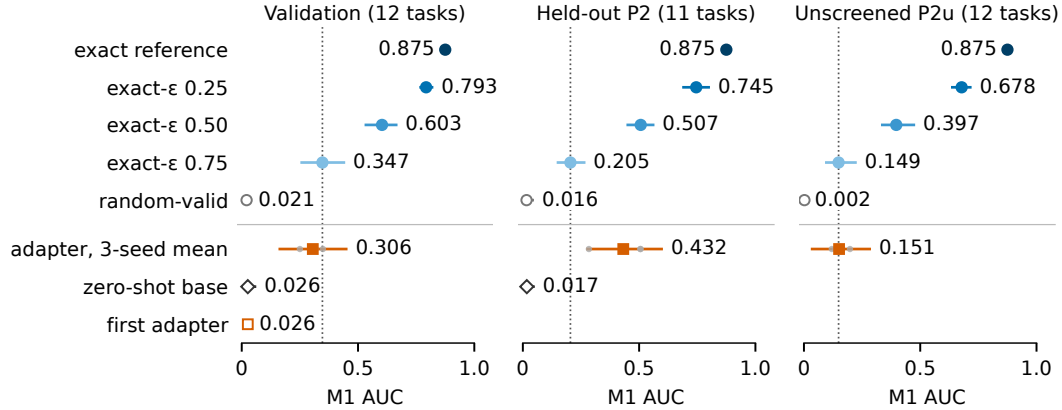


Figure 7: **M1 orders the privileged rungs on all three panels.** Points give M1, the area under the solve-versus-budget curve, with task-clustered 95% intervals, and faint points give the adapter’s three seeds. Only on the held-out panel (P2) does the adapter’s interval lie entirely right of the dotted $\epsilon=0.75$ line. The rungs read privileged information, so the ladder validates the measurement, not a policy. The unscreened ladder (P2u) is descriptive.

Table 16: **Frozen validation panel membership.** Random-valid reached the goal in some but not all screening episodes on every task. R gives uncapped exact-reference expansions (greedy / w3), and Screen gives random-valid goals in the screening episodes.

Task	C^*	R	Screen
15puzzle-935006	8	46 / 12	1
blocksworld-935001	10	26 / 26	4
blocksworld-935004	10	13 / 13	2
depot-935005	10	14 / 15	1
depot-935011	9	13 / 13	1
elevators-935000	8	11 / 11	2
elevators-935013	8	11 / 11	2
grid-935016	8	9 / 9	4
storage-935000	10	20 / 17	1
storage-935001	8	18 / 15	2
towers_of_hanoi-935000	13	24 / 25	1
towers_of_hanoi-935002	15	24 / 26	2

Three documented deviations affect how the panel was generated and rendered, not which tasks it holds. First, generation reuses the frozen candidate generator, which reproduces all 12 snapshots byte-for-byte although domain generators take the seed differently. Second, a first concurrent generation run corrupted the blocksworld outputs through a shared temporary file. That run was discarded, all candidates were regenerated serially and reproduced, and the deviation was disclosed before evaluation. Third, only the two additive native references seed the pre-rendered views, rather than four algorithms, because native BFS exceeds the 128-expansion reference ceiling on these tasks, and unseen states are rendered live.

Q.4 SCALED ADAPTER

The validated measurement places one trained policy above random-valid and two policies at its level (step 4 of Box 1). The trained adapter’s D_3 , the three-seed mean of its M1 minus random-valid’s, is $+0.285$ on the validation panel and $+0.416$ on the held-out panel, and both intervals exclude zero (Table 18). On the validation panel every seed is positive, as the pre-registered rule requires.

The adapter’s ladder position depends on the panel. It is separated above the 0.75 rung only on the held-out panel. On the other two panels and pooled, its difference from that rung is not separated from zero, which is not equivalence, because no equivalence margin was pre-registered for this contrast (Table 18). On the validation panel it also falls below the 0.50 rung (descriptive, Table 21).

Table 17: **Development-stage validation-panel arms.** M1 falls in order from the exact reference down the ϵ rungs to random-valid. M1 has task-clustered intervals, M2 and M3 are solved-only, and the rule selectors use privileged information unavailable to the policy.

Arm	M1 [95% CI]	$2\times$	M2 ρ	M3 κ
Exact reference	0.875 [0.875, 0.875]	1	1.0	1.021
Exact- ϵ 0.25	0.793 [0.764, 0.824]	0.992	1.089	1.024
Exact- ϵ 0.50	0.603 [0.528, 0.670]	0.95	1.246	1.032
Exact- ϵ 0.75	0.347 [0.252, 0.445]	0.717	1.374	1.023
Random-valid	0.021 [0.007, 0.038]	0.083	1.692	1.042
hadd-greedy	0.882 [0.774, 0.955]	0.958	0.962	1.083
bfs-order	0.000 [0.000, 0.000]	0	undefined	undefined
Novelty-first selection	0.021 [0.000, 0.052]	0.167	1.909	1.000
worst-first	0.000 [0.000, 0.000]	0	undefined	undefined
First adapter	0.026 [0.010, 0.042]	0.208	1.810	1.050
Scaled adapter (seed 17)	0.349 [0.172, 0.531]	0.583	1.326	1.140
Pretrained base	0.000 [0.000, 0.000]	0	undefined	undefined

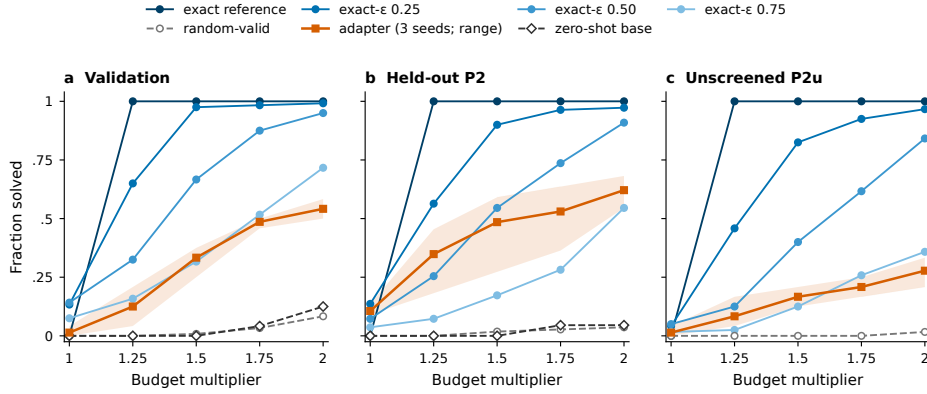


Figure 8: **Solved fraction against decision budget.** On all three panels the privileged rungs fall in order between the reference and random-valid, and the trained adapter’s area exceeds random-valid’s. Panels are validation, held-out (P2), and unscreened (P2u, descriptive), and the zero-shot base appears where measured. The adapter band is its three-seed range, not a confidence interval. Areas under these curves are M1, and the privileged rungs test only their own reference–random axis.

The held-out separation rests on 11 task clusters, where percentile intervals can under-cover, and per-task differences are negative on 3 of them.

Two other policies score at the random-valid level, which shows that on these panels valid choices alone do not produce a gain. The pretrained base, prompted zero-shot under the same contract, observation, and cap, makes only valid choices. It is nonetheless equivalent to random-valid on the validation panel within the pre-registered ± 0.05 margin (one development-stage realisation), and its confirmatory held-out verdict is inconclusive. On both panels it falls below the 0.75 rung and below the adapter (Appendix Q.4.4). The first adapter, an earlier small-corpus adapter, also scores at the random-valid level on the validation panel (Table 17).

Without the screen, the gain survives but rests on 4 of 12 tasks. All 12 unscreened tasks happen to have no random-valid screening goals, and on the 8 tasks that do not carry the gain, neither the adapter nor random-valid solves (Table 18). This is descriptive evidence that the gain survives without the screen, not evidence about where choice discriminates.

The gain is not carried by a single domain. Over all three panels, 22 of 35 per-task differences are positive and one is negative, and leaving out any one domain of the validation and held-out panels keeps every D_3 and its lower bound positive (descriptive, Figure 9, Table 22).

One on-policy DAgger round, run for one seed on half the planned corpus, does not separate the adapter from the 0.75 rung (Appendix Q.4.3). Together these results support a development-stage

Table 18: **The adapter beats random-valid on every panel but clears the 0.75 rung only on the held-out panel.** Development-stage three-seed adapter differences in M1 with paired task-clustered 95% intervals. Yellow rows are descriptive. ✓ denotes separation above zero and = denotes non-separation, not equivalence. The last column counts tasks with a positive, zero, or negative adapter-minus-random-valid difference (descriptive, not task-level tests).

Panel	Tasks	D_3 vs. random-valid		S vs. $\epsilon=0.75$		Tasks +/0/-
Validation	12	+0.285 [+0.140, +0.433]	✓	-0.041 [-0.187, +0.102]	=	9/2/1
Held-out (P2)	11	+0.416 [+0.250, +0.591]	✓	+0.227 [+0.053, +0.403]	✓	9/2/0
Unscreened (P2u)	12	+0.149 [+0.030, +0.283]	✓	+0.002 [-0.145, +0.157]	=	4/8/0
Pooled	35	—		+0.058 [-0.040, +0.156]	=	22/12/1

Table 19: **Descriptive seed-17 per-task adapter gain over random-valid.** The gain is uneven across tasks, large on both storage and both elevators tasks and zero on both blocksworld tasks. The pre-registered three-seed endpoint is reported in Table 21.

Task	D	Task	D
15puzzle-935006	-0.0625	elevators-935013	0.6875
blocksworld-935001	0.0	grid-935016	0.3625
blocksworld-935004	0.0	storage-935000	0.7375
depot-935005	0.0625	storage-935001	0.7
depot-935011	0.7125	towers_of_hanoi-935000	0.2375
elevators-935000	0.5	towers_of_hanoi-935002	0.0

choice-quality result relative to random-valid and the exact- ϵ ladder, on three panels with three training seeds. They carry no planning-ability or generality claim.

The rest of this subsection gives the adapter’s frozen protocol, corpus, recipe, and per-seed results, followed by the held-out panels, illustrative episodes, the DAGger ablation, and the zero-shot policy.

Each part of the adapter protocol was frozen before the step it governs: the protocol before task generation, the corpus before training, and the schedule before launch. The endpoint D is the mean over training seeds of the adapter’s M1 minus random-valid’s M1, with a task-cluster interval (10,000 draws, seed 133). The verdict POSITIVE requires a positive lower bound, $D \geq 0.05$, and a positive estimate at every seed, and the equivalence margin is ± 0.05 . Controls are reused from the ladder test.

The training corpus shares no task with any evaluation panel, and its menu augmentation places the target last no more often than chance. Of 2,400 serially generated training tasks, 1,182 remained eligible after excluding overlap with every evaluation panel and deduplicating. The seed-major walk reached 2048 records per algorithm from 236 tasks in 12 domains. Two reseeded menu orders per record give 8,192 samples, with the target last in 17.58% of samples against a chance rate of 17.58%.

The adapter changes the first adapter’s data, not its recipe. The recipe matches the first adapter’s except for corpus, augmentation, sample order, and seed (LoRA rank 64, alpha 128, 128 steps, seed 17). The smoke gate accepted 34/34 calls, and all 24/24 learned episodes, capped at $2R$, replayed with no mismatch and no invalid operation. For seed 17, the $2\times$ solve fraction is 0.583, M2 ρ 1.326, and M3 κ 1.140. Over 691 multi-option decisions heap-head agreement is 0.210 against chance 0.125, the last-label rate is 0.205, and the first-label rate is 0.090.

Four documented choices shaped this run, and only the first bears on what it can show. First, a scope edit to one training seed left seed variance unmeasured in that run. A later pre-registered replication closed the gap by training seeds 29 and 71 with the byte-identical recipe and the seed-17 sample order, so the seeds change only the LoRA initialisation and dropout. All 48 of its new episodes were independently replayed with no mismatch. Second, the step budget matches the first adapter’s recipe. Third, the seed-major corpus walk resolves an ambiguity in the protocol text. Fourth, both evaluation hooks first failed because the scheduler passes only the terminal path. After a fix that derives the attempt directory from that path, the audit re-ran on the same attempts and passed without affecting GPU work or episodes.

On the validation panel the adapter beats random-valid at every seed, but it sits near or below the exact- ϵ 0.75 rung. Per-seed D is +0.328 [+0.155, +0.508] for seed 17, +0.229 [+0.108, +0.354]

for seed 29, and $+0.297$ [$+0.134, +0.466$] for seed 71, so every lower bound is positive. Per-seed M1 is 0.349, 0.250, and 0.318, with a between-seed standard deviation of 0.051 (descriptive). The pre-registered difference from the 0.75 rung is $S = +0.002, -0.097$, and -0.029 per seed. Against the 0.50 rung the three-seed S is -0.298 [$-0.468, -0.129$] (descriptive), below the rung. Table 21 sets these values beside the held-out and unscreened panels.

The adapter also picks the last menu label less often than the first adapter did. Its last-label rate per seed is 0.205, 0.192, and 0.171 (chance about 0.13, descriptive), against 0.300 (chance 0.144) for the first adapter on the same panel.

Q.4.1 HELD-OUT PANELS

This subsection shows that both fresh panels were frozen before any evaluation episode and gives every arm’s results on them. The held-out panel appears as P2 in tables and the unscreened panel as P2u.

The held-out panel repeats the validation panel’s construction on fresh seeds. Both fresh panels come from 2,400 candidates generated from seeds 955000–955199. Removing overlap with earlier panels (1,168 candidates), duplicates (205), goals at the initial state (31), and C^* out of range (536) left 460. The held-out panel then applies the validation panel’s generation, C^* , and screening rules, keeping a candidate when $8 \leq C^* \leq 20$ and random-valid reaches the goal in 1 to 9 of its 10 screening episodes. The freeze walk, sorted by domain and seed, takes at most 2 tasks per domain up to 12 tasks, with a minimum of 8 tasks from 5 domains. Only 11 admitted tasks were available under the per-domain limit, so the panel froze with 11 tasks from 8 domains. That meets the minimum, so every held-out test is confirmatory.

The unscreened panel drops the random-valid screen, so it shows descriptively whether the gain survives without it. It takes kept candidates outside the held-out panel on which the exact reference reaches the goal under both algorithms, at most 2 per domain, without consulting the screen. This gives 12 tasks from 7 domains, and all 12 happen to have 0/10 random-valid goals in screening. All 638/638 held-out-panel and 696/696 unscreened-panel zoo episodes were independently replayed, with none missing.

The seed range was extended once, before any evaluation episode, with every other rule unchanged. The first freeze, from the original 40-seed pool 955000–955039, admitted 6 tasks from 5 domains and took the protocol’s blocked exit. At that point only controls-only screening had run. The amendment extended the seeds to 955000–955199, kept the 480 earlier candidates unchanged, and generated the new seeds 955040–955199 serially after them. The generation mechanism, the exclusion sources, the duplicate check, the C^* rule, R_t , the screen, the freeze walk, and the unscreened-panel rule all stayed fixed, and both panels were frozen from the combined pool.

The ladder test passes on the held-out panel and, descriptively, on the unscreened panel. On the held-out panel the four pre-registered adjacent gaps, from the exact reference down to random-valid, are $+0.130$ [$+0.070, +0.190$], $+0.239$ [$+0.183, +0.290$], $+0.302$ [$+0.259, +0.340$], and $+0.189$ [$+0.136, +0.248$], so every lower bound is positive. On the unscreened panel the descriptive gaps are $+0.197$ [$+0.154, +0.243$], $+0.281$ [$+0.191, +0.355$], $+0.248$ [$+0.209, +0.290$], and $+0.147$ [$+0.088, +0.226$], and every lower bound is again positive. Table 20 gives each arm’s M1 on all three panels.

Every seed’s gain over random-valid has a positive lower bound on every panel, but the rung separations need care. Table 21 gives D per seed and D_3 with intervals on each panel and pooled over 35 tasks, together with the separation S from the exact- ϵ rungs. Seed 29 has the lowest per-seed D on every panel, and on the held-out panel its separation point from the 0.75 rung is $+0.080$ (descriptive). No equivalence margin was pre-registered for S , so a non-separated S is not equivalence to the rung. The held-out separation rests on 11 task clusters, with 8 positive and 3 negative per-task differences, and percentile intervals can under-cover at this size.

Over the validation and held-out panels no single domain carries the gain, while on the unscreened panel it comes from four of twelve tasks. Table 22 counts tasks by the sign of their three-seed difference and leaves out one domain at a time over the validation and held-out panels. Over the 35-task union 22 tasks are positive, 12 are zero, and 1 is negative. On the unscreened panel the eight zero tasks are both 15-puzzle, both blocksworld, both depot, and both driverlog tasks, on which

Table 20: **Descriptive M1 by panel and arm.** On every panel the adapter’s three-seed mean lies above random-valid and below the exact reference. Values have task-clustered 95% intervals, and the exact- ϵ rungs and hadd-greedy use privileged information the policy never sees.

Panel	Arm	M1 [95% CI]
Validation (12)	Exact reference	0.875 [0.875, 0.875]
	Exact- ϵ 0.25	0.793 [0.764, 0.824]
	Exact- ϵ 0.50	0.603 [0.528, 0.670]
	Exact- ϵ 0.75	0.347 [0.252, 0.445]
	Random-valid	0.021 [0.007, 0.038]
	Pretrained base	0.000 [0.000, 0.000]
	Adapter, seed 17	0.349 [0.172, 0.531]
	Adapter, seed 29	0.250 [0.125, 0.375]
	Adapter, seed 71	0.318 [0.151, 0.490]
	Adapter, three-seed mean	0.306 [0.158, 0.455]
	hadd-greedy (privileged)	0.882 [0.774, 0.955]
P2 (11)	Exact reference	0.875 [0.875, 0.875]
	Exact- ϵ 0.25	0.745 [0.685, 0.805]
	Exact- ϵ 0.50	0.507 [0.445, 0.566]
	Exact- ϵ 0.75	0.205 [0.145, 0.269]
	Random-valid	0.016 [0.000, 0.048]
	Pretrained base	0.000 [0.000, 0.000]
	Adapter, seed 17	0.506 [0.313, 0.699]
	Adapter, seed 29	0.284 [0.119, 0.489]
	Adapter, seed 71	0.506 [0.295, 0.705]
	Adapter, three-seed mean	0.432 [0.273, 0.602]
	hadd-greedy (privileged)	0.928 [0.883, 0.966]
P2u (12)	Exact reference	0.875 [0.875, 0.875]
	Exact- ϵ 0.25	0.678 [0.632, 0.721]
	Exact- ϵ 0.50	0.397 [0.331, 0.478]
	Exact- ϵ 0.75	0.149 [0.091, 0.227]
	Random-valid	0.002 [0.000, 0.006]
	Pretrained base	0.000 [0.000, 0.000]
	Adapter, seed 17	0.198 [0.031, 0.396]
	Adapter, seed 29	0.120 [0.026, 0.224]
	Adapter, seed 71	0.135 [0.031, 0.250]
	Adapter, three-seed mean	0.151 [0.030, 0.288]
	hadd-greedy (privileged)	0.906 [0.883, 0.933]

neither the adapter nor random-valid solves within $2R_t$. Every leave-one-domain-out D_3 lies from $+0.305$ to $+0.395$, and every lower bound is at least $+0.185$. All of these results are descriptive.

Q.4.2 ILLUSTRATIVE CHOICE-FRONTIER EPISODES

These rule-selected episodes illustrate what the adapter saw and selected on one solved validation task and one unsolved held-out task. On the validation task the adapter follows the reference on 7/9 comparable decisions and reaches the goal while random-valid exhausts its budget, and on the held-out puzzle every adapter seed exhausts the cap while exact search solves (Figures 10, 11, and 12). They are illustrations, not an efficacy estimate or a diagnosed failure mechanism, and reference heuristics are used only for analysis, never as policy input.

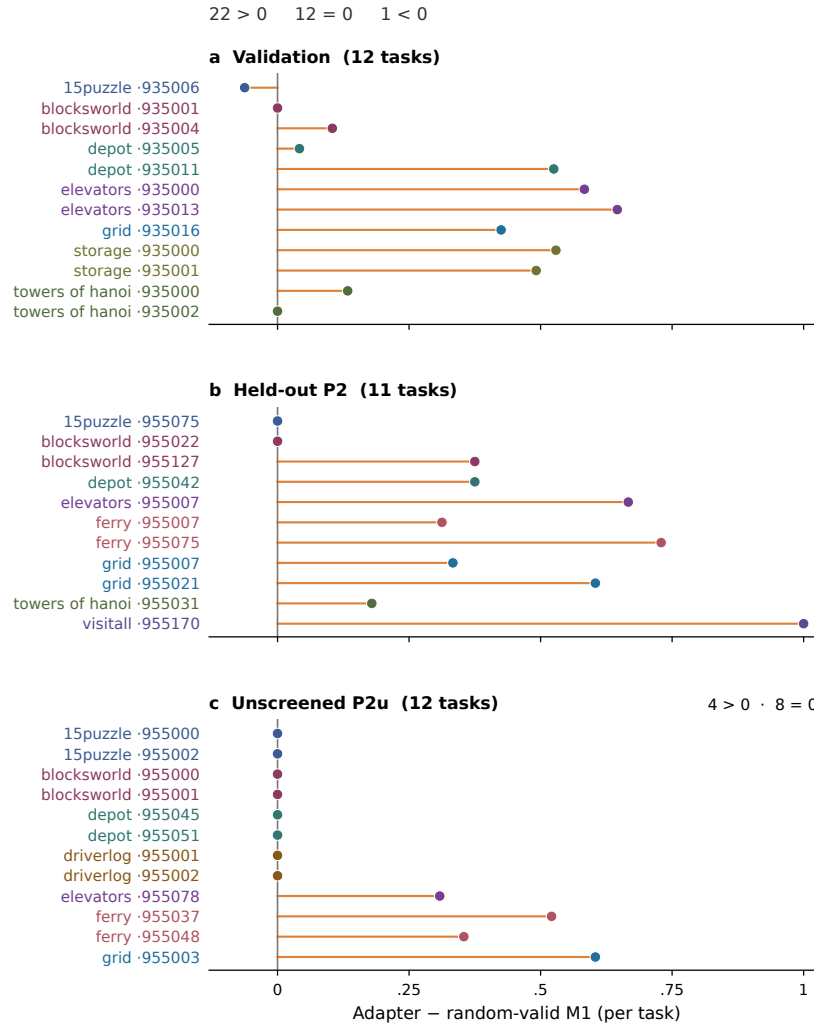


Figure 9: Descriptive task-level choice-frontier gains. Most tasks gain, and the unscreened panel’s improvement is concentrated on four tasks. Each point is the three-seed adapter mean minus random-valid M1, averaged across the two additive algorithms for a task, and colours identify domains. These points are not task-level significance tests.

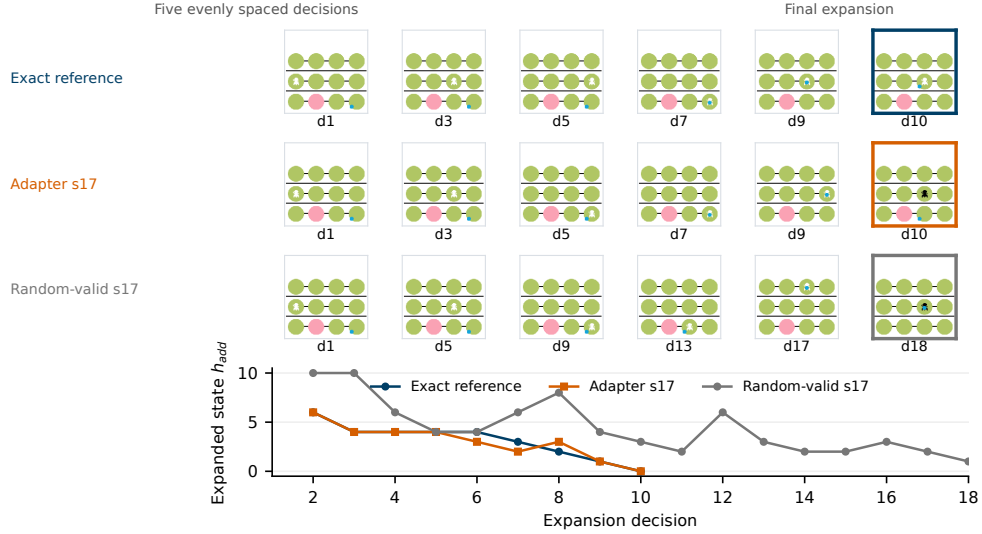
a Same task, different expansion choices

Figure 10: **On a solved validation task, the adapter follows the reference to the goal.** In this rule-selected weighted-A* episode, the seed-17 adapter agrees with the reference on 7/9 comparable decisions, and both reach the goal at decision 10, while random-valid exhausts its decision cap at 18. Rows show expanded scenes, and the plot traces their h_{add} after the first expansion. The example is illustrative, not an efficacy estimate.

b The entire largest menu (expansion decision 10)

Exact agreement 429/2117 vs chance 269,711 expected
Labels are the policy's opaque choices; h_{add} is privileged (not shown to policy).

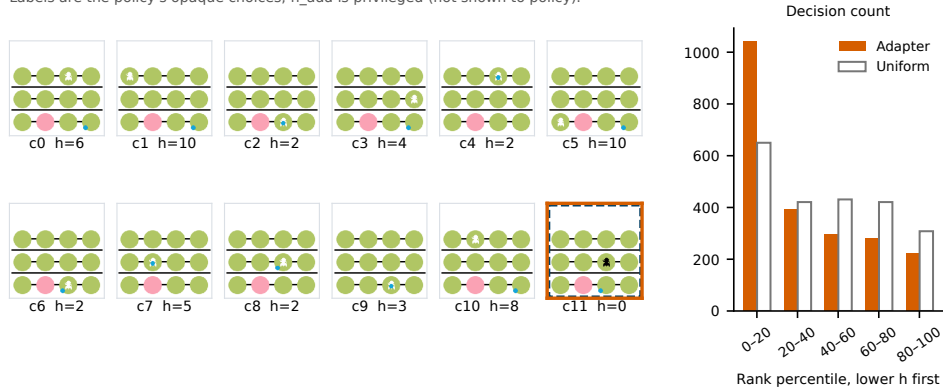


Figure 11: **The adapter picks the reference's state at the episode's largest menu and prefers low- h_{add} states overall.** At decision 10 of Figure 10, the adapter and the privileged reference both select c_{11} from 12 choices. The policy sees only opaque labels and scenes, not the heuristic values or the reference pick. The histogram describes the adapter's heuristic ranks across validation decisions against uniform choice on the same menus and is not an efficacy estimate.

Table 21: **Seed and rung contrasts on development panels.** The adapter separates above random-valid on every panel, and above the 0.75 rung only on the held-out panel. Estimates are paired task-clustered 95% intervals for adapter minus random-valid (D) and minus a privileged ϵ rung (S), and yellow rows are descriptive. ✓ means separated above, = not separated, and ▼ separated below.

Panel	Contrast	Estimate [95% CI]	Verdict
Validation (12)	D , seed 17	+0.328 [+0.155, +0.508]	
	D , seed 29	+0.229 [+0.108, +0.354]	
	D , seed 71	+0.297 [+0.134, +0.466]	
	D_3	+0.285 [+0.140, +0.433]	✓
	S vs. $\epsilon=0.75$	-0.041 [-0.187, +0.102]	=
	S vs. $\epsilon=0.50$	-0.298 [-0.468, -0.129]	▼
P2 (11)	D , seed 17	+0.490 [+0.294, +0.688]	
	D , seed 29	+0.268 [+0.093, +0.477]	
	D , seed 71	+0.490 [+0.276, +0.694]	
	D_3	+0.416 [+0.250, +0.591]	✓
	S vs. $\epsilon=0.75$	+0.227 [+0.053, +0.403]	✓
	S vs. $\epsilon=0.50$	-0.075 [-0.247, +0.089]	=
P2u (12)	D , seed 17	+0.196 [+0.031, +0.396]	
	D , seed 29	+0.118 [+0.026, +0.220]	
	D , seed 71	+0.133 [+0.031, +0.248]	
	D_3	+0.149 [+0.030, +0.283]	✓
	S vs. $\epsilon=0.75$	+0.002 [-0.145, +0.157]	=
	S vs. $\epsilon=0.50$	-0.246 [-0.400, -0.082]	▼
Pooled (35)	D , seed 17	+0.334 [+0.223, +0.450]	
	D , seed 29	+0.203 [+0.124, +0.290]	
	D , seed 71	+0.301 [+0.200, +0.411]	
	D_3	+0.279 [+0.191, +0.375]	✓
	S vs. $\epsilon=0.75$	+0.058 [-0.040, +0.156]	=

Table 22: **Descriptive concentration of three-seed gains.** Leaving out any one domain keeps D_3 positive with a positive lower bound. The upper rows count per-task signs, and the lower rows leave one domain out of the validation-P2 union.

Panel	Positive	Zero	Negative
Validation (12)	9	2	1
P2 (11)	9	2	0
P2u (12)	4	8	0
Union (35)	22	12	1
Domain left out	D_3	95% interval	Tasks left
15-puzzle	+0.384	[+0.269, +0.501]	21
blocksworld	+0.395	[+0.269, +0.520]	19
depot	+0.353	[+0.224, +0.484]	20
elevators	+0.305	[+0.185, +0.430]	20
ferry	+0.331	[+0.212, +0.455]	21
grid	+0.331	[+0.204, +0.464]	20
storage	+0.332	[+0.209, +0.459]	21
Tower of Hanoi	+0.384	[+0.262, +0.507]	20
visitall	+0.318	[+0.211, +0.422]	22

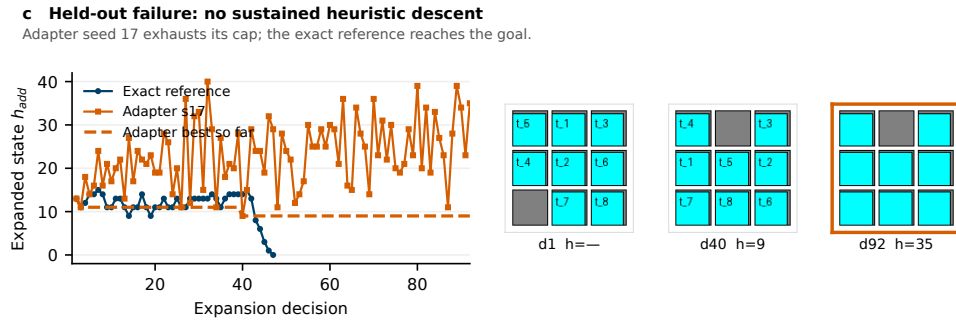


Figure 12: **On a held-out puzzle, the adapter stalls where the reference solves.** On this rule-selected puzzle, the reference reaches the goal at decision 47, while every adapter seed exhausts the cap at 92. The shown seed-17 adapter’s best-so-far h_{add} stops improving at 9 after decision 40. The first cached scene lacks tile glyphs that later scenes show. The trace is illustrative and does not establish why the adapter fails.

Q.4.3 DAGGER ABLATION

This subsection reports the pre-registered DAGger ablation, which finds no detectable change in the adapter’s choice quality after one on-policy round at one seed.

The ablation asks whether one round of on-policy data lifts the adapter above the exact- ϵ 0.75 rung. It collects on-policy records with the seed-17 adapter under each additive algorithm, aggregates them with replay records from the frozen corpus, retrains, and evaluates on the validation and held-out panels. The primary endpoint is the paired separation S of the DAGger adapter from the 0.75 rung pooled over the 23 tasks, with SEPARATED_ABOVE requiring a positive lower bound. The co-primary is the change Δ in M1 from the pre-DAGger adapter, with equivalence margin ± 0.05 .

A deadline cut the ablation to one seed and half the planned corpus, and the cut was made before any DAGger training or evaluation outcome. The amendment, at 11:21 UTC on 2026-09-24, kept only seed 17 because the three-seed plan could not fit a hard wall-clock deadline. The running seed-29 collection was cancelled with 1.79 GPU-h still charged, and seed 71 was not run. Each algorithm’s training set became the first 512 on-policy seed-17 records plus 512 replay records, which the menu augmentation turns into 2048 samples over 64 steps, with the recipe otherwise unchanged.

One DAGger round at one seed produces no detectable change in measured choice quality, so the ablation is inconclusive. The primary endpoint is NOT_SEPARATED, with $S = +0.115$ $[-0.026, +0.258]$ over 23 tasks. The co-primary is INCONCLUSIVE, with $\Delta = -0.030$ $[-0.114, +0.057]$ against the ± 0.05 margin. As descriptive secondary results, S is $+0.278$ $[+0.082, +0.473]$ on the held-out panel and -0.034 $[-0.197, +0.135]$ on the validation panel, and the last-label rate on the validation panel falls from 0.205 to 0.137. All 24/24 validation-panel and 22/22 held-out-panel DAGger episodes were independently replayed with no mismatch.

Q.4.4 ZERO-SHOT POLICY

This subsection shows that the untrained base model, prompted under the same contract, observation, and cap, is not separated from random-valid.

The zero-shot policy is the pretrained Qwen3-VL-8B-Instruct base with no adapter, prompted to choose by the goal. It receives the frozen scene-only choice-frontier observation and one appended system message, which states the JSON format and asks for the frontier state whose scene appears closest to satisfying the goal pages. The policy uses a fixed three-rule output extractor, greedy decoding at inference seed 17, and the adapter’s cap of $2R_t$ calls. It ran on both additive algorithms over the validation and held-out panels, 46 episodes in all.

A trained second backbone was ruled out before any outcome because it did not fit the compute cap. The preferred arm, InternVL3.5-8B trained with the frozen three-seed recipe, has a mean observation of 11,748 tokens against 3,111 for Qwen3-VL, which puts training alone at 43 to 55 GPU-h against the 30 GPU-h cap. A reduced recipe would break the byte-identical condition, so the arm was not run in any form. The protocol was committed before any GPU launch, and no amendment followed.

The zero-shot base is not separated from random-valid and falls below both the 0.75 rung and the trained adapter, as pre-registered (Table 23). Here D_3 names the pre-registered learned-minus-random-valid endpoint for both arms, a single realisation for the zero-shot arm. Validation-panel verdicts are development-stage and held-out-panel verdicts are held-out confirmatory. On the held-out panel the upper bound of D_3 exceeds the ± 0.05 margin by 0.001, so the frozen rule returns INCONCLUSIVE rather than EQUIVALENT. The arm’s M1 is 0.026 [0.000, 0.063] on the validation panel, 0.017 [0.000, 0.051] on the held-out panel, and 0.022 [0.003, 0.046] pooled, below the exact- ϵ 0.75 rung on every panel. The pre-registered prediction, D_3 within the margin with S and the adapter contrast A separated below on both panels, held.

The base always emits a valid choice, but its choices agree with the reference’s heap head at about the chance rate. All 1,592 model calls produced a valid choice through the first extractor rule. Heap-head agreement minus chance is $+0.003$ on the validation panel and $+0.001$ on the held-out panel, the last-label rate is 0.328 and 0.285 against a chance rate of about 0.13 to 0.14, and 4 of 46 episodes solve at $2R_t$ (descriptive). All 46/46 episodes were independently replayed with no mismatch.

Table 23: **Zero-shot base contrasts by panel.** The base is not separated from random-valid and falls below both the 0.75 rung and the trained adapter on every panel. Estimates are paired task-clustered 95% intervals with descriptive exact sign-flip p values, and pooled rows are descriptive. $\equiv$ applies the pre-registered ± 0.05 margin, $?$ is inconclusive, $\blacktriangledown$ separated below, and $|D_3| < 0.05$ is a descriptive point-only comparison.

Panel	Contrast	Estimate [95% CI]	Verdict	p
Validation (12)	D_3 vs. random-valid	+0.005 [−0.025, +0.044]	$\equiv$	0.906
	S vs. $\epsilon=0.75$	−0.321 [−0.422, −0.226]	$\blacktriangledown$	4.88×10^{-4}
	A vs. adapter	−0.280 [−0.425, −0.142]	$\blacktriangledown$	0.004
P2 (11)	D_3 vs. random-valid	+0.001 [−0.048, +0.051]	$?$	1.000
	S vs. $\epsilon=0.75$	−0.188 [−0.239, −0.140]	$\blacktriangledown$	9.77×10^{-4}
	A vs. adapter	−0.415 [−0.589, −0.248]	$\blacktriangledown$	0.004
Pooled (23)	D_3 vs. random-valid	+0.003 [−0.024, +0.033]	$ D_3 < 0.05$	0.896
	S vs. $\epsilon=0.75$	−0.257 [−0.324, −0.197]	$\blacktriangledown$	2.38×10^{-7}
	A vs. adapter	−0.344 [−0.459, −0.232]	$\blacktriangledown$	7.63×10^{-6}

R EXTERNAL AUDITS OF PUBLISHED INTERFACES

The identity audit passes on ScienceWorld (Wang et al., 2022), a published sequential-choice interface, so it serves as a positive control for step 1 of Box 1. Random-valid diverges from the gold trajectory at the first decision of every audited pair and solves no task, against 0.915 for the reference, so success there measures choice. The audit covers 6 of ScienceWorld’s 30 task types. None of the seven audited-candidate interfaces we surveyed uses an enumeration contract, so the 48/48 identity is shown only on our runtime (Table 24).

LLM-First Search (Herr et al., 2025) shows by construction why step 2 counts the budget in decisions. There the model scores candidates that a search then orders, and our audit replaces the paper’s GPT-4o with a local open-weight substitute, Qwen3-30B-A3B-Instruct. Random-valid diverges from the reference by the third decision in every pair, yet under the paper’s token caps it solves every task. A token cap never binds a chooser that spends no tokens, and because the frontier keeps every alternative, such a chooser is a complete randomized search. Counted in expansions, random-valid solves 0.0775 at the reference’s own count and 0.19 at twice it (Appendix R.1).

The rest of this appendix shows how ScienceWorld was chosen from a survey of published decision interfaces, and the LLM-First Search audit follows as a subsection.

The survey found no published evaluation that uses our enumeration contract, so the external audits run on interfaces where the model picks or scores candidates. It admitted published evaluations from 2022 on in which the model chooses from a runtime-supplied candidate set inside a planning or search loop, with public tasks and code. Seven candidates passed, five of type B (the model picks one valid action per step), two of type C (the model scores candidates that a search orders), and none of type A, our enumeration contract (Table 24). The closest published analogue, LFG (Shah et al., 2023), has the model score candidates in the enumeration style, but it was screened out because no evaluation harness was released. Four candidates are not CPU-feasible under their papers’ budgets. For WebShop (free-text search, no step limit) and LLM-First Search (an unreported token cap) this was a judgement call. LLM-First Search was the only candidate predicted to saturate, a prediction the later audit tested (Appendix R.1). ScienceWorld is the most cited feasible candidate, and the runner-up shares its prediction.

The surveyed interfaces differ in how they count budget, and only one counts a resource that random-valid does not spend. Six of the seven count budgets in steps, plan-length multiples, or iterations. Only LLM-First Search counts tokens, which a chooser that makes no model call never exhausts (Appendix R.1).

On ScienceWorld success measures choice, because the reference usually succeeds and random-valid never does. The frozen protocol takes the first 200 test variations under the paper’s 100-step limit with five random-valid seeds (17, 5077, 6131, 7409, and 8527), and ten re-run episodes are byte-identical. The reference completes 183 of 200 (0.915), including 3/9 boil, 6/9 melt, 4/9 freeze, 6/9 change-state, 135/135 use-thermometer, and 29/29 measure-melting-point variations. It falls

Table 24: **Surveyed decision interfaces and identity audits.** Both audited external interfaces register divergent choices, and random-valid matches the reference only on our runtime’s enumeration contract, whose serial tie-break rule causes the identity. B selects an action, C scores search candidates, and A enumerates candidates. “div.” means divergent and “sat.” predicts saturation. = means identical choices, while ✓ marks an observed divergence.

Interface	Venue	Type	CPU	Pred.	Audit	Budget
AutoPlanBench et al., 2025)	(Stein ICAPS 2025	B	✓	div.	—	plan length ×
ScienceWorld et al., 2022)	(Wang EMNLP 2022	B	✓	div.	✓ 1,000/1,000	100 steps
GLAM (BabyAI-Text)	ICML 2023	B	✓	div.	—	steps (code)
WebShop	NeurIPS 2022	B	✗	div.	—	no step limit
RAP	EMNLP 2023	C	✗	div.	—	iterations
LLM-First Search (Herr et al., 2025)	arXiv 2025	C	✗ [†]	sat. [†]	✓ 400/400	tokens
EmbodiedBench	ICML 2025	B	✗	div.	—	steps
Our enumeration (additive)	This paper	A	—	—	= 48/48	calls
Our choice frontier	This paper	—	—	—	✓ 18/18	2 <i>R</i> decisions

[†] Audited later with a GPU reference model; paper-budget saturation follows by construction (Appendix R.1). ✓/✗ in CPU denotes feasibility; “—” means no audit or no survey designation, not a negative result.

short only where 17 of the 36 Matter variations need more than 100 moves. Random-valid completes none, and 997 of its 1000 episodes end in a failure score.

The audit covers a narrow slice of ScienceWorld on a newer simulator version. The slice holds 6 of 30 task types (200 of 1,819 test variations), on all of which the paper’s own Random-Valid score is 0.00. The paper reports normalized scores, so its DRRN score of 0.084 on these tasks is not a success rate. The audit uses ScienceWorld 1.3.0, whose notes warn that gold paths can change relative to 1.2.x, and the paper’s simulator version was not re-run.

R.1 LLM-FIRST SEARCH AUDIT

A token budget cannot bind a chooser that spends no tokens. The LLM-First Search audit shows the consequence and why step 2 of Box 1 counts decisions instead.

LLM-First Search keeps every unexpanded alternative, and this makes a zero-token random chooser a complete search. It is a type-C interface. The model scores candidates, and the search expands the argmax child or pops the highest-valued frontier node, so every unexpanded alternative stays in the frontier. Random-valid answers every query with independent uniform draws, spends zero tokens, and makes no model call. It uses the five seeds of the first audit, and a decision is one node expansion after the root.

The audit runs the upstream code unchanged under the paper’s own budgets and declares four deviations. The upstream code (commit 3025bdaa, Apache-2.0) is imported unchanged and not vendored. The fixed rule audits every configuration whose published GPT-4o win rate exceeds 10%, which excludes Sudoku 6×6 and keeps Countdown 3, 5, and 7 and Sudoku 4×4, with games 0 to 19 of each, 80 tasks in all. Budgets are the paper’s per-game token caps (1,000,000 for Countdown and 100,000 for Sudoku 4×4). The deviations are these. The paper’s GPT-4o is replaced by the local Qwen3-30B-A3B-Instruct-2507, so reference success is that model’s. The reference runs once per task. Sudoku 6×6 is excluded. Random-valid uses a faster frontier implementation that produced byte-identical logs on 57/57 probe episodes.

Random-valid diverges from the reference in every pair, and early. No pair of the 400 is identical, and the first divergence has median index 0 and maximum 2. All 80/80 reference and 400/400 random-valid episodes were independently replayed with no mismatch.

Under the paper’s token budget random-valid nevertheless wins every episode, and this saturation follows by construction. Random-valid spends no tokens and the frontier keeps every alternative, so it is a complete randomized search, and a pre-audit probe had won 100/100 non-audited episodes.

Table 25: **Descriptive LLM-First Search budget audit.** Random-valid saturates under the paper’s token budget but falls far below the reference under a matched decision budget. Differences are substituted local reference minus random-valid success with task-cluster 95% intervals. The matched-budget reference success equals its paper-budget success by construction, and ✓ denotes registered choice and = saturation.

Configuration	Reference	Random-valid	Difference [95% CI]
<i>Paper token budget (=, saturated)</i>			
Countdown 3 (20)	1.000	1.000	0.000 [0.000, 0.000]
Countdown 5 (20)	0.750	1.000	−0.250 [−0.450, −0.100]
Countdown 7 (20)	0.650	1.000	−0.350 [−0.550, −0.150]
Sudoku 4×4 (20)	0.900	1.000	−0.100 [−0.250, 0.000]
All (80)	0.825	1.000	−0.175 [−0.263, −0.100]
<i>Matched decision budget (✓, choice registered)</i>			
Countdown 3 (20)	1.000	0.120	+0.880 [+0.770, +0.970]
Countdown 5 (20)	0.750	0.110	+0.640 [+0.370, +0.870]
Countdown 7 (20)	0.650	0.020	+0.630 [+0.410, +0.840]
Sudoku 4×4 (20)	0.900	0.060	+0.840 [+0.700, +0.950]
All (80)	0.825	0.0775	+0.747 [+0.648, +0.838]

Its only way to lose was a 500,000-expansion safety guard, which never bound. Random-valid used a median of 201 expansions and a maximum of 102,606, against a median of 29.5 and a maximum of 1,300 for the reference. Random-valid wins at every seed, so the verdict is SATURATED, and every reference loss ends at the token cap (Table 25).

A budget counted in decisions lets success register choice again. Within the reference’s own number of expansions per task, random-valid wins 0.0775 (per seed 0.0625, 0.075, 0.075, 0.0875, and 0.0875), so the verdict is CHOICE.REGISTERED. Doubling that budget raises random-valid’s success only to 0.19. The reference’s matched-budget success equals its token-budget success by construction, so the gap between the two arms in Table 25 is the evidence that choices matter. The matched-budget prediction held, and the paper-budget saturation follows by construction. Published GPT-4o success on these configurations is 0.768. One amendment, made before any audit episode, only reordered the smoke and audit start, and the ledger spent 2.89 of 12 GPU-h with no paid API use.

S RECEIPT TABLES

This appendix holds the gate receipts, the per-window ledgers, and the artifact index, so that a reader can trace each reported run to its receipt.

Every branch of the program reconciles with its ledger and replay, with zero missing evidence (Table 26). Five branches completed at v1 scope, while two stopped at frozen v1 cost-admission gates with terminal VALID.STOP receipts and later completed at reduced v2 scope. The table combines each branch’s frozen scope, gate status, coverage, and compute from the synthesis and follow-up ledgers, and its Tickets column indexes internal ticket numbers.

The Gate column uses one code per kind of outcome, because the branches ended in different ways. C is completion at declared scope, $V \rightarrow C$ a frozen v1 VALID.STOP followed by reduced-scope v2 completion, $H \rightarrow C$ the native arms’ v1 gate history followed by v2 completion, and R the reduced recovery reserve. Q_s denotes choice sensitivity under the choice-frontier contract, Q_r registered choice in the external audit, P the instrument-validation pass, X a failed counterfactual prediction, and $\blacksquare$ saturation under the LLM-First Search paper budget. ✓ marks a positive gate, = non-separation, $\equiv$ equivalence on the validation panel, and ? an inconclusive result on the held-out panel. In the GPU-hour column, the second-backbone cap reflects the prospective 12.11 GPU-h reserve transfer, and corruption and seed replication share the existing program caps. Stress-test R1–R4 spend covers both stress rows, and all follow-up windows have independent ledgers.

The program ledger charges failed attempts as well as successful ones, so its total does not hide unsuccessful runs. At the synthesis snapshot, the program recorded 135 attempts (96 succeeded, 36

Table 26: **Every branch reconciles with zero missing evidence, and each follow-up window keeps its own GPU-hour ledger.** Scope gives bindings or episodes and independently replayed counts. Follow-up windows are not summed into the program total. Gate codes are explained below.

Branch	Tickets	Gate	Scope / replay	GPU-h / cap
Expanded baseline	#116–#118	C	1,152 bindings; 576 model; 1,152/1,152	6.53 / 56
Dagger	#78–#84	C	405 episodes; 81 paired; 405/405	10.91 / 48
Successors	#85–#89, #99	C	90 episodes; 15 cells; 90/90	10.85 / 64
Curriculum	#119	C	243 model; 324 controls; 567/567	11.78 / 48
Transfer	#104–#107	C	39 cells (12 v1, 27 v2); 39/39	3.60 / 24
Backbone v2	#101–#103, #123	$V \rightarrow C$	72 model; 72 controls; 144/144	6.03 / 52.11
Generalization	#96, #124	#98, $V \rightarrow C$	1,200 (300 model, 900 controls); 1,200/1,200	6.04 / 32
Corruption	#126	C	360/360 new	shared
Seed replication	#127	C	534/534 new	12.79 / —
Recovery reserve	—	R	—	0.00 / 11.89
Program total				68.52 / 336
<i>Follow-up windows (independent ledgers)</i>				
Native arms	#128, #129	$H \rightarrow C$	216/216 new; 72/72 controls	3.2086 / 12
Stress R1/R2/R4	#130	C	72 menu; 80 variants; 18 cross-cell; 170/170	7.7998 / 36
Backbone R3	#130	C	72/72 model; 72/72 controls	shared
Choice frontier	#132	Q_s	144/144 panel; 58 preparations	1.069 / 12
Comparator zoo	#133	C	432/432 bindings; 342 new	CPU
Counterfactual	/ #134	$\times$	48 pairs; 288 counts; 96 sequences	CPU
M4				
Validation	#135	P	696/696 CPU; 48/48 model	1.19 / 4
Scaled adapter	#136	$\checkmark$	24/24 learned	4.84 / 30
External audit	#137	Q_r	1,000 pairs; 200 tasks	CPU
Replication / panels	#138, #139	$\checkmark$	48/48 new; zoo 638/638 P2, 696/696 P2u; learned 88/88 P2, 96/96 P2u	15.22 / 24
Dagger ablation	#140	$=$	24/24 validation; 22/22 P2	9.06 / 40
Zero-shot	#141	$\equiv / ?$	46/46 model	2.25 / 30
LFS audit	#142	$\blacksquare$	80/80 reference; 400/400 random-valid	2.89 / 12

failed, 3 cutoff), and failed and cutoff attempts retain their hours in the ledger. Every attempt ended before the absolute scheduler cutoff of 2026-09-21T11:55:19Z.

Two later addenda brought the program total to 68.52/336 GPU-h. The observation-corruption suite added 2.22 GPU-h to the generalization branch, and the training-seed replication addendum (closed 2026-09-21) added 12.79 GPU-h and 534 independently replayed episodes covering two cells. Appendix F reports the seed-variance results.

Follow-up windows. Each follow-up window has its own cap and ledger, none is added to the program total, and every window finished under its cap. Failed attempts retain their hours in every window. The native-arms window (v1 probes, then v2 training and evaluation) spent 3.2086 of 12 GPU-h and independently replayed its 216 new bindings (144 model episodes and 72 CPU random-valid episodes) and 72 reused comparators. The R1-R4 follow-up window spent 7.7998 of 36 GPU-h across two ledgers. Its cap was resized from 20 GPU-h before any outcome, because its frozen worst-case estimate of 27.04 GPU-h including spend exceeded the initial cap, and realized cost landed at 29% of that estimate. The choice-frontier window spent 1.069 of 12 GPU-h (training 0.81, smoke 0.052, model evaluation 0.208), with corpus derivation and controls on CPU. It replayed its 144 panel episodes and re-derived its 58 preparation episodes bit-exact. The comparator zoo launched no GPU job and replayed all 432 zoo bindings, of which 342 are new CPU episodes and 90 are reused $2 \times$ random-valid controls. The three-seed replication and the held-out and unscreened panels shared one window, which spent 15.22 of 24 GPU-h. The Dagger ablation window spent 9.06 of 40 GPU-h, including 1.79 GPU-h charged for a cancelled collection. The zero-shot window spent 2.25 of 30 GPU-h, and the LLM-First Search audit spent 2.89 of 12 GPU-h on its reference server with no paid API use.

Table 27: **Each follow-up claim class traces to pinned evidence files.** Filenames are relative to each directory prefix and the evidence repository, and claim classes identify their use.

Relative file / directory	Claim class
outputs/native-arms/v1/ evaluation/: evaluation.json, analysis.json	Clean / corruption contrasts
identity-audit.json	Enumeration identity
stress-evaluation.json	Menu / R2 / replay
r2-decomposition.json	Termination modes
budget.json; R1–R4 v1/v2 ledgers	Window accounting
outputs/expanded-study/v1/ second-backbone-v2/evaluation/ second-backbone-v3/evaluation/	Backbone BFS Backbone additive
outputs/choice-frontier/ v1/evaluation/ (evaluation, identity-audit, analysis, final-audit)	Gate / panel / replay
v1/preparation/visual-choice-frontier/ (re- port, audit)	Corpus derivation
v1/budget.json	Window accounting
o4/zoo/manifest.json	Zoo coverage / replay
o4/metrics/ (analysis, stored-posthoc-analysis, optimal- costs, native-four-algorithm-reference-costs)	Zoo M1–M5 / Holm / costs
v4/seeds/metrics/analysis.json	Three-seed validation
v4/panels/metrics/analysis.json	P2 / P2u / concentration
v5/metrics/analysis.json	DAGger ablation
v4/budget.json; v5/budget.json	Window accounting
v6/metrics/analysis.json	Zero-shot policy
v6/budget.json	Window accounting
outputs/external-audit/v2/ lfs/identity-audit.json	LFS identity
budget.json	External v2 ledger
docs/experiments/ and ticket #133	
native-arms/issue-129-closeout.md; issue-130-closeout.md	Native / stress receipts
choice-frontier/issue-132-closeout.md	Binding erratum
choice-frontier/ metric-adjudication-2026-09-23.md	Metric adjudication
Ticket #133 closing comment	Zoo numbers / boundary

Artifact index. Each follow-up claim class in the paper traces to a pinned evidence file, and Table 27 gives the mapping. Paths are relative to the evidence repository.

T RESULTS DETAIL: GATE RECEIPTS AND QUALIFICATION GATES

This appendix gives the program’s receipts, the follow-up ledgers, and the qualification gates that the accounting paragraph of Section 4 summarizes.

The expanded program stayed well inside its compute cap, and every executed branch reconciles with zero missing evidence. It ran on 2 A100 80GB GPUs under a 336 GPU-hour cap with an absolute scheduler cutoff, and spent 68.5244 of those hours in total, 53.52 (15.9%) at the synthesis snapshot and the remainder on the corruption and seed-replication addenda. Of the seven admitted branches, five completed at v1 scope. The other two, the second backbone and generalization/robustness, stopped at frozen v1 cost-admission gates with terminal VALID_STOP receipts and then completed at reduced v2 scope under new versioned protocols. Independent replay covers 1,152/1,152 baseline episodes, 405/405 DAGger episodes, and 90/90 successor-prediction episodes. Table 26 gives frozen scope, status, coverage, and ledger spend per branch.

The follow-up windows ran on their own ledgers and are never summed with the main program. Most used separate GPU ledgers, and three ran on CPU only (the comparator zoo, the counterfactual audit, and the first external audit). Each stayed within its cap: the native-arms window spent 3.2086

of 12 GPU-hours, the R1-R4 follow-up window 7.7998 of 36, and the choice-frontier window 1.069 of 12. The R1-R4 cap was raised from 20 to 36 GPU-hours before any outcome, because its frozen worst-case estimate of 27.04 GPU-hours including spend exceeded the initial cap. Realized cost landed at 29% of that estimate. Each follow-up closed with complete coverage and independent replay (Table 26).

Three earlier gates fixed which algorithms the active matrix contains. The first, the bounded BFS pilot, showed executable BFS with zero gain over the strongest control and therefore closed with a governed `VALID_STOP`. On a frozen outcome-blind 15-task BFS development panel, final checkpoints from five process-SFT seeds achieved 1.0 invariant-valid success with a 0.0 invalid-operation rate, while the base achieved 0.0 and random-valid achieved 1.0, so downstream BFS work was gated off. The BFWS structural gate then passed on its own frozen development panel (Table 4), although the overlap between the BFWS and BFS development panels is not pinned. The companion manifest of the frozen 45-task final evaluation was fixed before outcomes and has never been accessed.

The third gate retired the optimal A* arm over $h_{\max}$ with landmark counting after its own governed `VALID_STOP`. The two additive best-first variants supersede it, and its original heuristic-representation estimand remains unidentified. The five-seed BFS pilot above predates the budget rule, which authorizes one training run at seed 17 per enumeration-contract model and cell (Section 4).

U RETAINED-INFRASTRUCTURE FEASIBILITY NOTES

This appendix defines the verified infrastructure components behind corpus generation, evaluation, and evidence production. They are feasibility evidence for the runtime and rendering pipeline and cannot support a Search Process Policy efficacy claim.

Plan pipeline. **Plan Submission** transmits a supplied plan from the project adapter to a planning or rendering backend. **Plan Interpretation** converts the submitted plan into the backend’s ordered action representation. An **Action Sequence** is the canonical identity of a supplied plan as a normalized, ordered sequence of grounded action predicates. Casing and whitespace are not part of its identity.

Rendering pipeline. **Render Production** creates a visualisation output and its rendered image for a planning state. **Render Validation** confirms that the produced visualisation and image satisfy declared structural and semantic checks. **Plan Provenance** links a produced visualisation to the exact supplied plan that the backend interpreted.

Certification and evidence. **Integration Certification** is the finding that Plan Submission, Plan Interpretation, Plan Provenance, Render Production, Render Validation, and required execution-isolation checks all passed. An **Attempt** is one authorized, immutable execution with its own identity and retained outcome, whether successful or not. An **Evidence Bundle** is the self-contained records from an Attempt that allow an independent reproducer to verify its claims offline. The retained infrastructure supplies deterministic compact delta/event evidence with semantic replay while preserving search semantics, serving the replay-determinism requirement of the Trusted Search Runtime.

Historical terms. Planning Certificate, Joint Action-and-Certificate SFT, Adaptive Scaffolding, Support Route, Live Memory, Verified Joint Step, and Route Label are historical CGAS terms for retained infrastructure internals. They cannot support any current Search Process Policy efficacy claim and are not referenced elsewhere in this manuscript.

V DECISION CONTRACTS

The two decision contracts differ in what the policy emits and what the runtime owns, so their results are not comparable. Table 28 sets them side by side. Section 3 defines the enumeration contract, and Appendix Q.1 defines the choice-frontier contract.

Table 28: **Two non-comparable decision contracts.** The policy emits and the runtime checks different units under the enumeration and choice-frontier interfaces. The identity-audit row records the executed scope, not a cross-contract performance comparison.

Attribute	Enumeration	Choice frontier
Policy output	Grounded, unscored Typed Search Operation	Opaque frontier label
Frontier owner	Search Memory; visited / score	Runtime; seeded per-decision labels
Observation	Modality adapter	Visual: task / initial / 128px frontier / goal
Learned budget	2× reference decisions; base: 1 call	2× reference expansions and decisions; goal free
Identity audit	Diagnostic, additive cells	Pre-registered, 18 pairs

W USE OF LLM STATEMENT

Large language model agents drafted and revised the manuscript text and wrote the figure scripts under the authors’ direction. An agent pass searched for citations, and a human verified every cited source. Agent reviewers ran repeated review rounds on the drafts. LLM agents also wrote and ran experiment and analysis code, drafted protocols and two pre-evaluation protocol amendments on the authors’ behalf, and compiled evidence sheets, and the authors approved each protocol before its outcome. The authors verified every reported number against its pinned artifact and take full responsibility for the content.